Computación Distribuida con ZeroC Ice



Universidad de Castilla-La Mancha

Escuela Superior de Informática
de Ciudad Real

David Villa
Francisco Moya
Óscar Aceña

2026/10/28

Escuela Superior de Informática

e-mail	esi@uclm.es
Teléfono	926 29 53 00
Web	http://www.esi.uclm.es

© Los autores del documento. Se permite la copia, distribución y/o modificación de este documento bajo los términos de la licencia de documentación libre GNU, versión 1.1 o cualquier versión posterior publicada por la *Free Software Foundation*, sin secciones invariantes. Puede consultar esta licencia en http://www.gnu.org.

Contents

Contents iii

Prólogo ix

1. **Hola mundo distribuido (versión Python)** 1
   1.1. Especificación de interfaces 2
   1.2. Sirviente 3
   1.3. Servidor 4
   1.4. Referencias remotas 5
   1.5. Cliente 6
   1.6. Ejecutando el servidor 7
   1.7. Ejecutando el cliente 8
   1.8. Modos de invocación 9
   1.9. Objetos con estado 9
      1.9.1. Un contador por clave 10
      1.9.2. Un contador por identidad 10
   1.10. Ejercicios 12

2. **Transparencia de localización** 15
   2.1. `Registry` y `Locator` 16
   2.2. Creando proxies indirectos 18
   2.3. Invocando un proxy indirecto 19
   2.4. ¿Qué ocurre realmente? 19
   2.5. Ventajas 20

3. **Gestión de aplicaciones distribuidas (versión Python)** 21
3.1. Introducción 21
3.2. IceGrid 21
3.2.1. Componentes de IceGrid 22
3.2.2. Configuración de IceGrid 23
3.2.3. Arrancando IceGrid 25
3.3. Creando una aplicación distribuida 25
3.4. Instanciación del servidor 29
3.4.1. Instanciando un segundo servidor 30
3.5. Ejecutando la aplicación 31
3.5.1. Despliegue de la aplicación 31
3.5.2. Ejecutando los servidores 32
3.5.3. Ejecutando el cliente 34
3.6. Objetos bien conocidos 35
3.7. Activación y desactivación implícita. 36
3.8. Depuración 36
3.8.1. Actualizando los servidores 36
3.8.2. Descripción de la aplicación 37
3.9. Receta 37
3.10. Ejercicios 38

4. **Difusión de eventos (versión Python)** 39
4.1. Arranque del servicio 41
4.2. Federación de canales 43
4.2.1. Propagación entre canales de eventos federados 44

5. **Invocación y despachado asíncrono (versión Python)** 47
5.1. Invocación síncrona de métodos 47
5.2. Invocación asíncrona de métodos. 48
5.2.1. Recogiendo el resultado más tarde 48
5.2.2. Utilizando un *callback* 50

5.3. Despachado asíncrono de métodos 51
5.4. Mecanismos desacoplados 54
5.5. Ejercicios 54

6. **Patrones (versión Python)** 57
6.1. Contenedor de objetos 57
6.2. Factoría de objetos 60
6.3. Factoría IceGrid 64
6.4. Default servant 67

7. **Replicación** 69
7.1. Grupo de réplicas 70
7.2. Registry replicado 73

Referencias 75

Listado de acrónimos

AMI	Asynchronous Method Invocation
AMD	Asynchronous Method Dispatching
API	Application Program Interface
ARCO	Arquitectura y Redes de Computadores
ASM	Active Servant Map
CORBA	Common Object Request Broker Architecture
DDNS	Dynamic DNS
DDS	Data Distribution Service
DHCP	Dynamic Host Configuration Protocol
DNS	Domain Name System
DoS	Denial of Service
DTO	Data Transfer Object
Ice	Internet Communications Engine
IceP	Ice Protocol
IP	Internet Protocol
JMS	Java Message Service
NIC	Network Interface Controller
POSIX	Portable Operating System Interface; UNIX
RMI	Remote Method Invocation
RPC	Remote Procedure Call
Slice	Specification Language for Ice
SPoF	Single Point of Failure
SSH	Secure Shell
STL	C++ Standard Template Library
TCP	Transmission Control Protocol
TCP/IP	Arquitectura de protocolos de Internet
UDP	User Datagram Protocol
UUID	Universally Unique Identifier
XML	Extensible Markup Language
TLS	Transport Layer Security

Prólogo

Este documento es una introducción muy práctica al desarrollo de aplicaciones distribuidas mediante middlewares orientados a objeto. En concreto se utiliza el middleware Internet Communications Engine (Ice) de la empresa ZeroC, Inc. Se trata de un middleware de propósito general, que a pesar de estar diseñado por una empresa, se ciñe en gran medida a los estándares de la industria. Maneja básicamente los mismos conceptos y abstracciones del estándar Common Object Request Broker Architecture (CORBA), aunque es más sencillo, potente y flexible que la mayoría de las implementaciones disponibles para éste último.

Todas las técnicas y mecanismos explicados se complementan con ejemplos que utilizan este middleware. Son ejemplos de código intencionadamente simples, pero completamente funcionales. Por ello resultan perfectos para diseñar y desarrollar aplicaciones más complejas tomándolos como punto de partida.

A pesar de estar centrado en una tecnología concreta como es ZeroC Ice, prácticamente la totalidad de conceptos, servicios y mecanismos abordados son perfectamente extrapolables a otras tecnologías presentes en el desarrollo de aplicaciones en red y en los sistemas distribuidos pasados, presentes y probablemente futuros.

Tanto la documentación como el código ha sido desarrollado o adaptado por los miembros del grupo ARCO[1]. Es el resultado de la gran experiencia del grupo derivada tanto de docencia universitaria en redes de computadores y sistemas distribuidos como de la participación en importantes proyectos de investigación y desarrollo tanto públicos como privados a lo largo de varias décadas.

Sobre los ejemplos

Todos los ejemplos que aparecen en este documento (y algunos otros) están disponibles para descarga a través del repositorio git en:

`https://github.com/uclm-esi/hello.ice`

Aunque es posible descargar estos ficheros individualmente o como un archivo comprimido, se aconseja utilizar el sistema de control de versiones git[2].

Si encuentra alguna errata u omisión es los programas de ejemplo, por favor, utilice la herramienta de gestión de incidencias (*issue tracker*) accesible desde:

[1] `http://arco.esi.uclm.es`
[2] `https://git-scm.com/`

https://github.com/uclm-esi/hello.ice/issues

Es importante que el lector disponga en su computador de una copia actualizada de este repositorio ya que es necesario para seguir correctamente muchas de las explicaciones y ejemplos. En los listados de código y de ejecución de comandos se incluyen rutas que corresponden a los directorios de este repositorio. Por ejemplo, al mostrar la ejecución del siguiente comando, se debe entender que se realiza desde el subdirectorio `py/hello` del repositorio de ejemplos:

```
py/hello$ ./server.py
```

Sobre este documento

Los fuentes de este documento (en LaTeX) también se encuentran en un repositorio git https://github.com/uclm-esi/ice-book aunque no es accesible públicamente. Si quiere colaborar activamente en el desarrollo o mejora de este documento póngase en contacto con los autores.

Al igual que los ejemplos, también existe una herramienta de gestión de incidencias (pública) en la que puede notificar problemas o errores de cualquier tipo que haya detectado en el documento.

https://github.com/UCLM-ESI/ice-book-public/issues

Chapter 1

«Hola mundo» distribuido

[Python]

Ice traslada el mecanismo de invocación remota al modelo de objetos. El servidor no ofrece simplemente un conjunto plano de procedimientos, sino objetos sobre los cuales el cliente puede invocar sus métodos. Para lograrlo se usan las piezas habituales en cualquier middleware de este tipo: un lenguaje de especificación de interfaces, un compilador que genera a partir de él el código de cliente y de servidor, y un núcleo de ejecución (*runtime*) que serializa cada invocación, la transporta y devuelve el resultado.

La figura 1.1 sitúa estas piezas en una aplicación cliente-servidor típica. En el cliente, el compilador genera un *proxy*, un objeto local con la misma interfaz que el remoto que actúa como su representante local[1]. Cuando el programa invoca un método sobre el proxy, éste serializa la llamada y la envía al servidor. Allí, el *sirviente* —la clase que realmente implementa la funcionalidad del servicio— atiende la petición, pero antes de llegar a él la petición pasa por dos niveles de despachado: primero el **adaptador de objetos** localiza, a partir de la identidad que lleva la petición, qué sirviente debe atenderla; después el propio sirviente localiza, a partir del nombre de la operación, qué método invocar. Completada la operación, el resultado (o una excepción) recorre el camino inverso hasta el cliente.

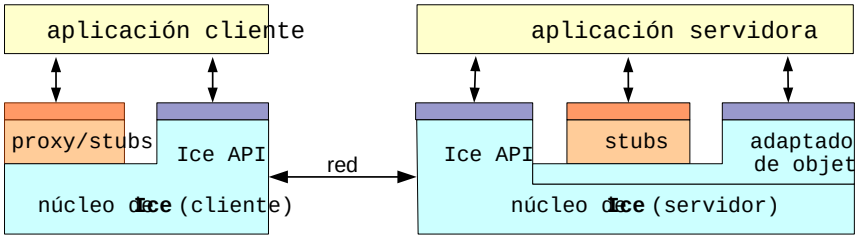


Figure 1.1: Componentes de una aplicación Ice

[1] A veces denominado *subrogate*

El diagrama de secuencia de la figura 1.2 muestra este recorrido completo para una invocación síncrona, es decir, aquella en la que el cliente queda bloqueado hasta recibir la respuesta.

Ice utiliza además un protocolo de aplicación propio, Ice Protocol (IceP), que puede transportarse sobre TCP, UDP o TLS, entre otros.

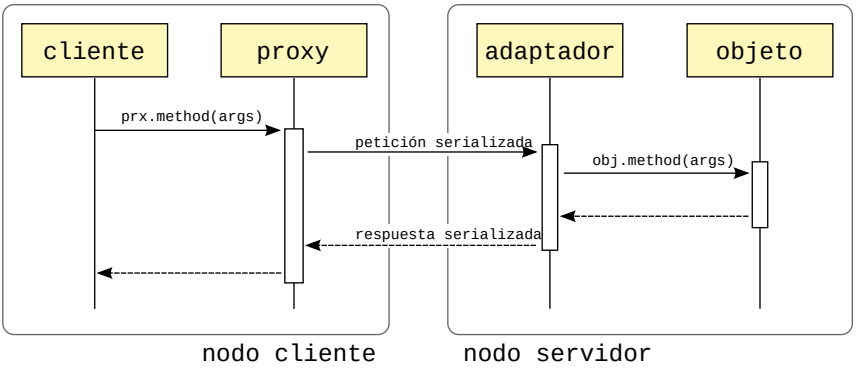


Figure 1.2: Diagrama de secuencia de una invocación remota síncrona

En este primer ejemplo construiremos un servicio *Printer* con un único método remoto, `write()`, que imprime en la salida estándar del servidor la cadena que el cliente le pase como parámetro. Lo usaremos para recorrer, paso a paso y con código, cada una de las piezas que acabamos de describir.

1.1 Especificación de interfaces

Antes de escribir el cliente o el servidor, es imprescindible fijar el contrato entre ambos. Ese contrato dice qué operaciones ofrece el objeto remoto, con qué parámetros y qué devuelven. Se escribe con Specification Language for Ice (Slice), el lenguaje de especificación de interfaces de Ice. Es un lenguaje puramente declarativo; no permite expresar lógica, solo tipos y operaciones.

El siguiente listado declara la interfaz *Printer*, con una única operación `write()` que recibe una cadena y no devuelve nada.

Listado 1.1: Especificación Slice de la interfaz *Printer*
py/hello/printer.ice

```
module Example {
  interface Printer {
    void write(string message);
  };
};
```

Además de interfaces, Slice permite declarar estructuras, secuencias, diccionarios, enumerados y excepciones, y las interfaces admiten herencia (incluso múltiple). También es posible marcar una operación como `idempotent` para indicar que el

runtime puede reintentarla sin riesgo si falla la comunicación; lo veremos en acción más adelante, en § 1.9.

Los compiladores de interfaces (`slice2cpp`, `slice2java`, `slice2py`…) traducen esta especificación al lenguaje elegido, generando los stubs de cliente y de servidor. Las reglas que rigen esa traducción para cada lenguaje se llaman *mappings*; por ejemplo, una secuencia SLICE se convierte en un `vector` de la STL en C++ y en un `array` en Java. La figura 1.3 resume ese esquema. Una misma especificación produce código distinto para cada lenguaje, pero todos son compatibles entre sí en tiempo de ejecución, así que un cliente escrito en un lenguaje puede usar objetos implementados en otro.

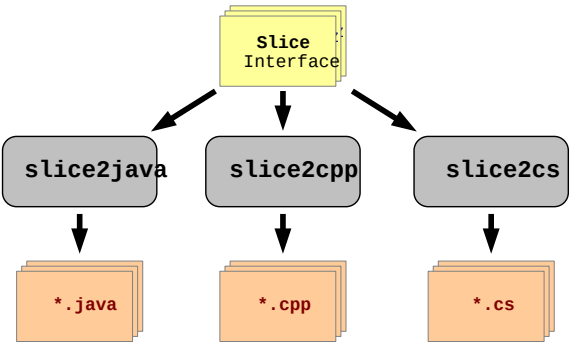


FIGURE 1.3: ICE genera un compilador de interfaces para cada lenguaje soportado

Python también dispone de un compilador, `slice2py`, pero lo habitual es compilar la interfaz al vuelo. Tanto el servidor como el cliente llaman al arrancar a la función `Ice.loadSlice()` con la ruta al fichero `printer.ice`, y esta genera en memoria el módulo `Example`, con el proxy y el stub de servidor. Justo después ya se puede importar:

```
Ice.loadSlice('printer.ice')
import Example
```

El ejemplo `py/counter-by-id` usa en cambio `slice2py` desde su `Makefile`.

La figura 1.4 muestra qué ficheros escribe el programador (en amarillo) y cuáles genera el compilador (en verde) para el cliente y el servidor, en la variante con `slice2py`. Los veremos en las secciones siguientes.

1.2 Sirviente

Un objeto ICE es una entidad conceptual; tiene un tipo (la interfaz que implementa) y una **identidad** que lo distingue de cualquier otro objeto del mismo servidor. El código que le da cuerpo —la clase, escrita por el programador, que implementa esa interfaz— se llama **sirviente**. La diferencia es importante. Un mismo sirviente puede responder por varios objetos, técnica que veremos en el patrón *default ser-*

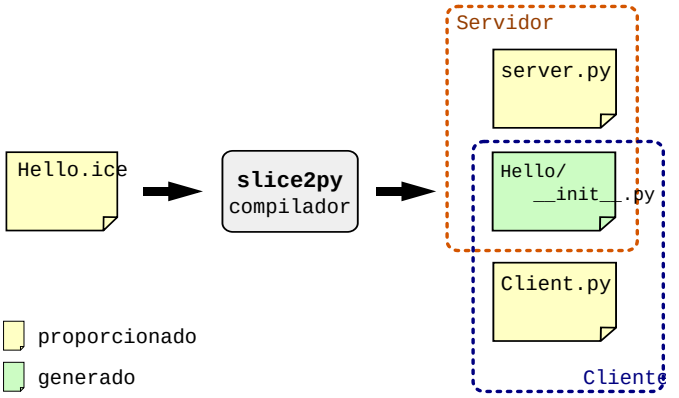


Figure 1.4: Esquema de compilación de cliente y servidor en Python

vant (§6.4), y un mismo objeto puede estar respaldado por sirvientes en varios servidores, lo que permite construir sistemas redundantes (capítulo §7).

El compilador de interfaces genera la clase *Example.Printer*. El sirviente hereda de esa clase y redefine cada uno de los métodos declarados en la interfaz Slice.

La implementación del sirviente (extremadamente simple) se muestra en el siguiente listado:

Listado 1.2: Sirviente de la aplicación Printer
py/hello/server.py

```
1  class PrinterI(Example.Printer):
2      n = 0
3
4      def write(self, message, current=None):
5          print("{0}: {1}".format(self.n, message))
6          sys.stdout.flush()
7          self.n += 1
```

1.3 Servidor

El **adaptador de objetos** es el componente del servidor que relaciona identidades con sirvientes. Al registrar un sirviente (método `add`) se le asocia una identidad, y el adaptador guarda esa asociación en una tabla. Al llegar una petición, el adaptador busca en esa tabla a partir de la identidad que lleva la petición y, una vez localizado el sirviente, éste despacha la operación a partir de su nombre. Son los dos niveles de despachado mencionados al principio del capítulo.

Nuestro servidor consiste principalmente en una función `main()` que recibe como parámetro el `communicator`[2], ya inicializado. Veamos el código en detalle:

[2]El `communicator` representa el *broker* de objetos del núcleo de comunicaciones: http://en.wikipedia.org/wiki/Object_request_broker

Listado 1.3: Servidor de la aplicación Printer
py/hello/server.py

```
1  def main(ic):
2      servant = PrinterI()
3      adapter = ic.createObjectAdapter("PrinterAdapter")
4      proxy = adapter.add(servant, ic.stringToIdentity("printer1"))
5
6      print(proxy)
7
8      adapter.activate()
9      ic.waitForShutdown()
10     return 0
11
12 if __name__ == "__main__":
13     try:
14         with Ice.initialize(sys.argv) as communicator:
15             sys.exit(main(communicator))
16     except KeyboardInterrupt:
17         print("\nShutting down server...")
```

En la **línea 2** se crea el sirviente (una instancia de la clase `PrinterI`). En la **línea 3** se crea el adaptador de objetos. Necesita saber en qué *endpoints* escuchar, pero esa información no está en el código, sino en la configuración, donde se busca a partir del nombre del adaptador, `PrinterAdapter`. Lo veremos al ejecutar el servidor (§ 1.6).

En la **línea 4** se registra el sirviente en el adaptador mediante el método `add()`, indicando para ello la identidad que tendrá el objeto (`printer1`). En este ejemplo la identidad es un identificador bastante simple, pero lo recomendable es utilizar una secuencia globalmente única (UUID). El método `add()` devuelve una referencia al objeto distribuido recién creado, que se denomina *proxy*.

La **línea 8** activa el adaptador. A partir de ese momento empieza a atender peticiones para sus objetos en sus propios hilos, mientras el hilo principal queda bloqueado en `waitForShutdown()` (**línea 9**) hasta que el comunicador termine.

Por último, el bloque `"__main__"` crea el comunicador con `Ice.initialize()` y se lo pasa a la función `main()`. Por hacerlo dentro de una sentencia `with`, el comunicador se destruye automáticamente al terminar. La excepción `KeyboardInterrupt` se captura para que el servidor termine con un mensaje cuando el usuario pulsa Control-C.

Con esto tenemos el servidor completo. Pero queda un problema. El cliente necesita un modo de encontrar el objeto alojado en el servidor. Ice proporciona mecanismos de localización (que veremos en capítulos posteriores), pero en este primer ejemplo haremos algo más sencillo y usaremos la representación textual del proxy que el propio servidor imprime al arrancar.

1.4 Referencias remotas

El proxy es la materialización local de una **referencia remota**, que incluye dos datos: la identidad del objeto y los *endpoints* del servidor que lo aloja (protocolo, dirección y puerto). Se obtiene al registrar el sirviente en el adaptador (§ 1.3),

como acabamos de ver. Veamos cómo se representa fuera del programa y cómo viaja como un valor más entre cliente y servidor.

Ice puede representar una referencia remota también como texto (*stringified proxy*):

```
printer1 -t:tcp -h 192.168.0.12 -p 7070
```

que corresponde con el formato:

```
object-identity options:endpoints
```

A su vez, los *endpoints* siguen el formato:

```
protocol -h host -p port
```

Las funciones `proxyToString()` y `stringToProxy()` convierten en ambos sentidos entre esta representación textual y el proxy que usa el programa, que funciona como una especie de «puntero remoto» al objeto. Gracias a la representación textual, el cliente puede obtener su primera referencia remota por cualquier medio —un fichero, un argumento en la línea de comandos, incluso un correo electrónico— sin necesidad de ningún servicio adicional. El capítulo 2 explica cómo evitar que esa referencia dependa de un *endpoint* concreto.

Como las referencias remotas son valores corrientes, se pueden pasar como argumento o devolver como resultado de una operación; en Slice el tipo de un proxy se escribe con un asterisco (`Printer*`). Es lo que hace, por ejemplo, una factoría de objetos (§ 6.2). El cliente invoca un método que crea un objeto nuevo en el servidor y le devuelve su referencia. Ice no incorpora un recolector de basura distribuido que libere esos objetos cuando ningún cliente los usa ya; es responsabilidad del programador, como se ve en ese mismo capítulo.

1.5 Cliente

La aplicación cliente únicamente debe conseguir una referencia al objeto remoto e invocar el método `write()`. El código completo del cliente aparece a continuación:

Listado 1.4: Cliente de la aplicación Printer
py/hello/client.py

```
1  def main(ic):
2      proxy = ic.stringToProxy(sys.argv[1])
3      printer = Example.PrinterPrx.checkedCast(proxy)
4
5      if not printer:
6          raise RuntimeError('Invalid proxy')
7
8      printer.write('Hello World!')
9
10 if __name__ == "__main__":
```

```
11      with Ice.initialize(sys.argv) as communicator:
12          main(communicator)
```

El programa acepta por línea de comandos la representación textual del proxy del objeto remoto. A partir de ella se obtiene un objeto proxy (**línea 2**), pero de un tipo genérico (`Ice.ObjectPrx`), la base de la que derivan todos los proxies. Para poder invocar los métodos de la interfaz *Printer* se requiere un proxy del tipo concreto (`PrinterPrx`), y para obtenerlo hay que comprobar que el objeto remoto efectivamente implementa esa interfaz. De eso se encarga el método `PrinterPrx.checkedCast()` (**línea 3**).

Esa comprobación no es gratuita. A diferencia de un *downcasting*[3] local, aquí solo el servidor sabe de qué tipo es realmente el objeto, así que `checkedCast()` implica una invocación remota que se lo pregunta. Preguntarle a un objeto por su propio tipo es una forma de **reflexión**, remota en este caso. La variante `uncheckedCast()` construye el proxy sin preguntar, a costa de asumir el riesgo si el objeto no es del tipo esperado.

Concretamente, toda interfaz Slice hereda de forma implícita de *Ice::Object*, que dota a cualquier objeto remoto de cuatro operaciones de introspección: `ice_ping()`, `ice_id()`, `ice_ids()` e `ice_isA()`. El molde comprobado no es más que una invocación de `ice_isA()` con el identificador del tipo (`::Example::Printer`) como argumento, de modo que, además de resolver el tipo, confirma de paso que el objeto existe y es alcanzable: si no implementa la interfaz se obtiene un proxy nulo, y si la identidad no corresponde a ningún objeto el servidor responde con `ObjectNotExistException`. Con `uncheckedCast()` ese diagnóstico se retrasa hasta la primera invocación, que fallará con `OperationNotExistException`. Y no siempre hay elección: comprobar exige una respuesta, así que sobre un proxy de solo petición (ver § 1.8) el molde comprobado ni lo intenta y falla de inmediato con `TwowayOnlyException`.

Una vez conseguido el proxy del tipo correcto (objeto `printer`) se puede invocar el método remoto `write()` (**línea 8**) pasando por parámetro la cadena `"Hello World!"`.

1.6 Ejecutando el servidor

Si se intenta ejecutar el servidor sin argumentos se obtiene un error:

```
py/hello$ python3 server.py
Traceback (most recent call last):
  ...
Ice.InitializationException: exception ::Ice::InitializationException
{
    reason = object adapter `PrinterAdapter' requires configuration
}
```

[3] Consiste en moldear (*cast*) un puntero o referencia de una clase a una de sus subclases, asumiendo que realmente es una instancia de ese tipo: http://en.wikipedia.org/wiki/Downcasting.

Como vimos en la sección 1.3, el servidor necesita información específica que le indique los *endpoints* en los que debe escuchar el adaptador `PrinterAdapter`. Esa información no está en el código. En Ice, la configuración de una aplicación —protocolos, puertos, tamaño de los *pools* de hilos, nivel de trazas, y muchas otras opciones— se ajusta por medio de **propiedades**, pares `clave=valor` que normalmente se escriben en un fichero de texto. El de nuestro servidor (`server.config`) aparece en el siguiente listado:

Listado 1.5: Configuración del servidor de Printer
py/hello/server.config

```
PrinterAdapter.Endpoints=tcp -p 7070
```

Esta propiedad en concreto indica que el adaptador debe utilizar un socket TCP en el puerto 7070. Al no especificar una IP, el adaptador escuchará en todas las interfaces de red del computador. El puerto tampoco es obligatorio. De no especificarlo, el adaptador elegiría automáticamente uno libre. Lo que sí es obligatorio es el protocolo que debe usar (en este caso TCP).

Los nombres de las propiedades se organizan en una jerarquía separada por puntos. Todas las que empiezan por '`Ice.`' afectan al propio middleware, y cada adaptador utiliza su propio nombre como prefijo, como en `PrinterAdapter.Endpoints`. Además de en un fichero, las propiedades se pueden fijar directamente en la línea de comandos. Gracias a esto, cambiar de protocolo, mover un servidor a otro puerto o activar una traza no requiere editar ni recompilar el código.

Para que el servidor cargue las propiedades del fichero de configuración, se indica su ruta con la opción '`--Ice.Config`':

```
py/hello$ python3 server.py --Ice.Config=server.config
printer1 -t -e 1.1:tcp -h 192.168.0.12 -p 7070 -t 60000:tcp -h 10.1.1.10 -p 7070 -t 60000
```

En esta ocasión el programa arranca correctamente y queda ocupando la shell como corresponde a cualquier servidor. Lo que aparece en consola es, como se indicó anteriormente, la representación textual del proxy para el objeto distribuido (§ 1.4). La identidad es '`printer1`'; el modo '`-t`' corresponde a *twoway* (volveremos sobre los modos de invocación en § 1.8); '`-e 1.1`' es la versión de la codificación de los mensajes; y el resto son los *endpoints*, separados por '`:`'. Hay uno por cada interfaz de red del computador, porque la configuración no fija ninguna dirección. Cada *endpoint* lleva su propio *timeout* en milisegundos ('`-t 60000`'), que no hay que confundir con el modo '`-t`' que sigue a la identidad.

1.7 Ejecutando el cliente

Para ejecutar el cliente debemos indicar en línea de comandos el proxy que imprime el servidor. Basta con la identidad y uno de los *endpoints*; las opciones que se omiten toman su valor por defecto. Nótese que se debe escribir entre comillas para que la shell lo interprete como un único parámetro:

```
py/hello$ python3 client.py "printer1 -t:tcp -h 192.168.0.12 -p 7070"
```

El programa se ejecuta y retorna inmediatamente. El servidor mostrará por consola la cadena `"Hello World!"`, precedida del número de mensajes recibidos hasta ese momento, como corresponde a la implementación del método `write()` del sirviente.

1.8 Modos de invocación

El modo de invocación no es parte de la interfaz, sino un atributo del proxy —el '`-t`' que acabamos de ver en § 1.6—, y determina qué intercambio de mensajes usa cada invocación y qué semántica obtiene el cliente (Cuadro 1.1).

Opción	Modo	Intercambio	Semántica
-t	*twoway*	petición y respuesta, sobre TCP	*at-most-once*
-o	*oneway*	solo petición, sobre TCP	*maybe*
-d	*datagram*	solo petición, sobre UDP	*maybe*

Table 1.1: Modos de invocación de un proxy Ice

El modo por defecto, *twoway*, es el único en el que el cliente recibe una respuesta, ya sea el resultado de la operación o una excepción si algo ha fallado. Los otros dos modos entregan la petición al transporte y devuelven el control inmediatamente, sin esperar nada; el cliente no tiene forma de saber si la operación llegó a ejecutarse, de ahí la semántica *maybe*. Por coherencia, solo se pueden invocar así las operaciones sin valor de retorno ni excepciones. Intentarlo con otra produce un error local (`TwowayOnlyException`) antes de enviar nada —lo comprobaremos en uno de los ejercicios del final del capítulo.

El modo *datagram*, al ir sobre UDP, añade además la posibilidad de que el mensaje se pierda o llegue desordenado; a cambio, un mismo *endpoint* puede ser una dirección multicast, lo que permite difundir una invocación a varios destinatarios a la vez.

Si una invocación *twoway* falla, el *runtime* de Ice la reintenta automáticamente cuando sabe con certeza que la petición no llegó a enviarse. Si la operación está marcada como `idempotent` (§ 1.1), la reintenta también cuando no puede saberlo, porque repetirla no cambia el resultado. Así preserva la semántica *at-most-once* del modo por defecto.

1.9 Objetos con estado

El sirviente `PrinterI` no guarda ningún estado entre invocaciones, y el servidor registra uno solo en el adaptador. Veamos ahora un servicio *Counter* con dos operaciones, `increment()` y `get()`, que sí necesita recordar un valor entre peticiones. Hay dos formas de organizarlo, y la comparación entre ambas es la mejor forma de entender para qué sirve la identidad de un objeto.

1.9.1 Un contador por clave

La primera es la que se haría con un RPC. Un único objeto remoto gestiona internamente varios contadores, y cada operación recibe como argumento adicional una clave que indica sobre cuál debe actuar.

Listado 1.6: Especificación Slice de *Counter*, con un argumento de clave
py/counter-by-key/counter.ice

```
module Example {
    exception NotFound {string name;}

    dictionary<string, long> CounterDict;

    interface Counter {
        void create(string name);
        void increment(string name, long delta) throws NotFound;
        long get(string name) throws NotFound;
        CounterDict list();
    }
}
```

El sirviente mantiene internamente un diccionario que asocia cada nombre con su valor, y cada operación empieza comprobando si ese nombre existe; si no, lanza la excepción *NotFound* declarada en la propia interfaz. El servidor registra un único sirviente en el adaptador, y el cliente debe indicar el nombre del contador en cada invocación. El código completo está en el repositorio de ejemplos, en `py/counter-by-key`.

1.9.2 Un contador por identidad

La segunda forma aprovecha el modelo de objetos. En lugar de un contador con muchas entradas, hay muchos objetos *Counter*, cada uno con su propio valor, y el cliente los distingue por su identidad en vez de por un argumento.

Listado 1.7: Especificación Slice de *Counter*, un objeto por contador
py/counter-by-id/counter.ice

```
module Example {
    interface Counter {
        long increment();
        idempotent long get();
    };
};
```

Nótese que `get()` está marcada como `idempotent`. Invocarla una vez o varias deja el contador exactamente igual, así que el *runtime* puede reintentarla aunque no sepa si llegó a ejecutarse (§1.8). Por otro lado, `increment()` no lo está, porque cada invocación que llega a ejecutarse cambia el resultado de la siguiente.

Listado 1.8: Servidor del servicio `Counter`, con dos sirvientes
py/counter-by-id/server.py

```python
class CounterI(Example.Counter):
    def __init__(self):
        self.value = 0

    def increment(self, current):
        self.value += 1
        return self.value

    def get(self, current):
        return self.value

def main(ic):
    adapter = ic.createObjectAdapter('CounterAdapter')

    proxy1 = adapter.add(CounterI(), Ice.stringToIdentity('counter1'))
    proxy2 = adapter.add(CounterI(), Ice.stringToIdentity('counter2'))

    print(proxy1)
    print(proxy2)

    adapter.activate()
    ic.waitForShutdown()
    return 0

if __name__ == '__main__':
    try:
        with Ice.initialize(sys.argv) as communicator:
            sys.exit(main(communicator))
    except KeyboardInterrupt:
        print("\nShutting down server...")
```

El servidor crea un sirviente `CounterI` por contador, y registra cada uno en el adaptador con una identidad distinta (`counter1` y `counter2`), igual que antes se registraba `printer1`. Son dos instancias de la misma clase, con su propio atributo `value`, tal y como dos instancias de una clase convencional tienen cada una su propio estado.

Listado 1.9: Cliente del servicio `Counter`
py/counter-by-id/client.py

```python
def main(ic):
    proxy = ic.stringToProxy(sys.argv[1])
    counter = Example.CounterPrx.checkedCast(proxy)

    if counter is None:
        raise RuntimeError('invalid proxy')

    print("get() = '{}'".format(counter.get()))
    print("increment() = '{}'".format(counter.increment()))
    print("increment() = '{}'".format(counter.increment()))
    print("get() = '{}'".format(counter.get()))

if __name__ == '__main__':
    with Ice.initialize(sys.argv) as communicator:
        main(communicator)
```

El cliente recibe en línea de comandos el proxy textual de uno de los dos contadores, y lo invoca cuatro veces. Al ejecutar primero el servidor y después el cliente dos veces seguidas sobre `counter1` y una tercera sobre `counter2` se obtiene:

```
py/counter-by-id$ ./server.py --Ice.Config=server.config
counter1 -t -e 1.1:tcp -h localhost -p 2002 -t 60000
counter2 -t -e 1.1:tcp -h localhost -p 2002 -t 60000
```

```
py/counter-by-id$ ./client.py 'counter1 -t:tcp -h localhost -p 2002'
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
py/counter-by-id$ ./client.py 'counter1 -t:tcp -h localhost -p 2002'
get() = '2'
increment() = '3'
increment() = '4'
get() = '4'
py/counter-by-id$ ./client.py 'counter2 -t:tcp -h localhost -p 2002'
get() = '0'
increment() = '1'
increment() = '2'
get() = '2'
```

Los dos proxies que imprime el servidor comparten los mismos *endpoints* —están en el mismo adaptador— pero tienen identidades distintas, y por eso `counter1` y `counter2` llevan cuentas independientes aunque compartan servidor, adaptador e interfaz. Con el diseño de la sección anterior, esto mismo habría requerido añadir un argumento a cada operación para indicar sobre qué contador actuar; aquí lo resuelve la identidad.

Estas ejecuciones, una tras otra, ocultan un aspecto del servidor que conviene tener presente, la concurrencia. El adaptador despacha las peticiones entrantes en los hilos de un *pool* cuyo tamaño por defecto es uno (propiedad `Ice.ThreadPool.Server.Size`), así que, aunque lleguen peticiones de varios clientes a la vez, el servidor las atiende una detrás de otra y `increment()` nunca se ejecuta dos veces a la vez sobre el mismo sirviente. Si se amplía el tamaño del *pool*, dos invocaciones a `increment()` sobre el mismo contador sí pueden solaparse, y entonces `self.value += 1` dejará de ser segura. Dos hilos pueden leer el mismo valor antes de que ninguno escriba el resultado, y se perderá un incremento. Lo veremos en los ejercicios.

1.10 Ejercicios

E 1.01 Modifica el cliente para que acepte una cadena por línea de comandos en lugar de enviar siempre `"Hello World"`. Comprueba que funciona.

E 1.02 Ejecuta varios clientes que envían cadenas distintas.

E 1.03 Con la herramienta más simple que conozcas, comprueba que el servidor efectivamente está escuchando en el puerto especificado.

E 1.04 Ejecuta servidor y cliente en computadores diferentes.

E 1.05 Modifica el sirviente para que acepte una cadena de texto en el constructor y la imprima junto con cada mensaje entrante.

E 1.06 Modifica el servidor para que registre en el adaptador varios objetos *Printer* con identidades distintas, pero con la misma instancia del sirviente para todos ellos. Imprime sus proxies e invócalos desde distintos clientes. ¿Puede el sirviente saber a qué objeto va dirigida cada invocación?

E 1.07 Intenta ejecutar el servidor en dos terminales distintos en el mismo computador ¿qué ocurre?

E 1.08 Edita el fichero `server.config` y modifica el endpoint del adaptador para que escuche en un puerto diferente. Comprueba que el cliente funciona correctamente con el nuevo puerto.

E 1.09 Elimina el puerto de la especificación del endpoint en `server.config` y comprueba que servidor y cliente siguen funcionando. Ahora puedes ejecutar dos servidores en el mismo computador ¿por qué?

E 1.10 Añade un endpoint UDP a `server.config`, arranca el servidor y ejecuta dos clientes que invocan el objeto utilizando cada uno un endpoint diferente. Escribe el comando `tshark`[4] que demuestra que efectivamente un cliente utiliza transporte TCP y el otro UDP.

E 1.11 Añade una pausa de 3 segundos al comienzo del método `write()` del sirviente. Ejecuta dos clientes en terminales diferentes tratando de que arranquen con la mínima diferencia de tiempo posible. A la vista del resultado ¿crees que el objeto está atendiendo a los clientes de forma concurrente? ¿Por qué?

E 1.12 Averigua cómo alterar los parámetros del proxy textual (el modo de invocación, el *timeout*…) mediante el API de Ice y escribe un programa que lo demuestre.

E 1.13 Crea una versión diferente del servidor. Éste, en lugar de imprimir el mensaje recibido en pantalla, lo enviará (añadiendo el prefijo «redirect:» al mensaje original) a otro objeto *Printer*. Para ello necesitarás el proxy de ese otro objeto, que puedes crear con la versión original del servidor. Utilizando el cliente original invoca a los objetos alojados en cada uno de los servidores.

E 1.14 Añade la propiedad `Ice.ThreadPool.Server.Size=4` al `server.config` del servicio *Counter* por identidad (§ 1.9) y ejecuta muchos clientes a la vez invocando `increment()` sobre el mismo contador (por ejemplo, con un bucle de shell que los lance en paralelo). Comprueba que el valor final no coincide con el número de invocaciones. ¿Por qué no ocurría esto con el tamaño de *pool* por defecto? Corrige el sirviente para que el resultado sea correcto con varios hilos.

[4] http://www.wireshark.org/docs/man-pages/tshark.html

E 1.15 Construye un proxy oneway del contador (`ice_oneway()`, § 1.8) e invoca `get()` sobre él. Comprueba qué excepción se produce y en qué momento —antes o después de contactar con el servidor—. Repite la prueba con `increment()` declarada `oneway` en el fichero `.ice` (sin valor de retorno) y comprueba que esta vez sí se puede invocar, aunque el cliente nunca sepa si realmente se ejecutó.

Chapter 2

Transparencia de localización

El direccionamiento es un mecanismo esencial en las redes de computadores y por tanto en cualquier aplicación distribuida Es simple: no se puede usar un servicio que no se puede encontrar.

En el capítulo «Hola mundo distribuido» hemos visto cómo desarrollar y ejecutar una aplicación distribuida básica. El servidor imprime por pantalla la **referencia remota** para el objeto que aloja como un *proxy textual*[1] como el siguiente:

```
printer1 -t -e 1.1:tcp -h 192.168.0.12 -p 7070 -t 60000
```

Esto corresponde con lo que Ice denomina *proxy directo*, es decir, una referencia remota que incluya toda la información necesaria para contactar con el servidor (TCP en este caso).

El cliente la utiliza para crear un objeto proxy (local) con el que invocar al objeto remoto. Sin embargo, desde el punto de vista de la gestión, resulta poco práctico para el cliente tener que conocer los detalles concretos de direccionamiento de los objetos.

Desde el punto de vista de la aplicación distribuida el proxy directo plantea varios problemas:

- Cambiar la dirección IP y el puerto invalida la referencia que se proporcionó al cliente. Las direcciones IP de un nodo pueden cambiar cuando se utiliza asignación dinámica con DHCP, que además es lo más común. El puerto también sería deseable que pudiera cambiar para lograr más flexibilidad, aunque no es tan importante.
- Incluso asumiendo que los nodos mantendrán su dirección IP y los servidores mantendrán su puerto, el proxy directo impediría mover el servidor a otro nodo.
- No sería posible añadir medios adicionales (*endpoints*) para contactar con el servidor.

[1] *stringified proxy*

En una aplicación de medio o gran tamaño se utilizan decenas o centenares de servidores y miles de objetos. Es obvio que hace falta un mecanismo más potente que pueda escalar sin problema.

Como en el caso de los servicios de Internet clásicos, la solución es utilizar *nombres* en lugar de direcciones, y proporcionar un sistema que puede traducir esos *nombres* a *direcciones* eventualmente *en el momento de realizar la invocación*. Esto es a lo que llamamos servicio de *binding*. Internet utiliza nombres de dominio y el servicio DNS. Ice utiliza *proxies indirectos* y el servicio `Locator`.

Un proxy indirecto no tiene información sobre la ubicación del servidor (no aparece ningún *endpoint*). En su lugar aparece un nombre simbólico asociado al adaptador que sirve el objeto. Veamos un ejemplo de *proxy indirecto* en formato textual:

```
printer1 -t -e 1.1 @ PrinterAdapter1
```

De esta forma Ice proporciona *transparencia de localización*, es decir, el mecanismo que permite que un programador (o usuario) pueda utilizar un objeto remoto sin necesidad de conocer su situación lógica ni física.

Esto posibilita que los servidores (objetos en nuestro caso) puedan migrar a cualquier otro lugar de Internet sin que los usuarios puedan percibir ningún cambio significativo. A su vez, la transparencia de localización habilita otras funcionalidades muy interesantes como redundancia, tolerancia a fallos o balanceo de carga, muy importantes en aplicación de gran escala o que requieren fiabilidad extra.

2.1 `Registry` y `Locator`

Ice dispone de un servidor denominado `registry` que proporciona múltiples servicios como veremos en próximos capítulos. Entre ellos, está el `Registry`, que permite registrar adaptadores de objetos, y el `Locator`, que permite hacer consultas para averiguar los *endpoints* de un adaptador previamente registrado[2].

Ambos son objetos Ice corrientes, con su interfaz declarada en Slice. El `Locator` es el que consulta el cliente:

```
1  interface Locator {
2      idempotent Object* findObjectById(Identity id)
3          throws ObjectNotFoundException;
4
5      idempotent Object* findAdapterById(string id)
6          throws AdapterNotFoundException;
7
8      idempotent LocatorRegistry* getRegistry();
9  }
```

Las dos operaciones de búsqueda devuelven un `Object*`, es decir, un proxy directo. Ahí está resumido el trabajo de un localizador: convertir un proxy indirecto en

[2]Como se puede comprobar la combinación de `Registry` y `Locator` se asemeja mucho a la funcionalidad de un DDNS solo que para objetos en lugar de nodos.

uno directo. Que ambas sean `idempotent` tampoco es casual, porque consultar no modifica nada y conviene que el *runtime* las pueda reintentar.

Hay dos granularidades de búsqueda, y se distinguen por cómo está escrito el proxy indirecto. Con `printer1@PrinterAdapter1` el cliente indica en qué adaptador vive el objeto, y el *runtime* invoca `findAdapterById()`. Con `printer1` a secas la referencia es solo la identidad, sin ninguna pista sobre su ubicación; entonces se invoca `findObjectById()` y se habla de un objeto «bien conocido». Esta última es la forma más desacoplada, y la que permite mover un objeto de un adaptador a otro.

El `Registry` es la cara complementaria, la que usan los servidores para darse de alta:

```
interface LocatorRegistry {
    idempotent void setAdapterDirectProxy(string id, Object* proxy)
        throws AdapterNotFoundException, AdapterAlreadyActiveException;
    ...
}
```

El programador no escribe esas invocaciones: las hace el adaptador de objetos al activarse, por el mero hecho de tener configurado un identificador y un `Locator`. La excepción `AdapterAlreadyActiveException` evita que dos servidores reclamen a la vez el mismo identificador.

Para arrancar una instancia del `registry` necesitamos un fichero de configuración, con una sintaxis equivalente a la que vimos en los servidores básicos, es decir, una lista de propiedades definida en forma «clave=valor».

Para una prueba sencilla podemos utilizar el siguiente fichero de configuración:

Listado 2.1: Configuración de un Registry
locator/registry.config

```
Ice.Default.Locator=IceGrid/Locator -t:tcp -h 127.0.0.1 -p 4061

IceGrid.Registry.Client.Endpoints=tcp -p 4061
IceGrid.Registry.Server.Endpoints=tcp
IceGrid.Registry.Internal.Endpoints=tcp
IceGrid.Registry.LMDB.Path=/tmp/icedata/registry

IceGrid.Registry.DynamicRegistration=1
```

La **línea 1** es la referencia al objeto `Locator`. Las **líneas 3–5** definen los *endpoints* en los que escuchan los distintos servicios del servidor. La **línea 6** indica dónde colocar la base de datos que utiliza el programa para guardar su estado, y por último, la **línea 8** activa la propiedad `DynamicRegistration` que permite registrar adaptadores de objetos desconocidos (necesario en este ejemplo).

Debemos crear el directorio referido (en el directorio `/tmp` por ser un ejemplo desechable):

```
$ mkdir -p /tmp/icedata/registry
```

Y para arrancar el servicio ejecutamos simplemente:

```
locator$ icegridregistry --Ice.Config=registry.config
```

Como es un servidor, el programa queda ocupando la consola indefinidamente.

2.2 Creando proxies indirectos

Por fortuna, los mecanismos que permiten a los servidores Ice utilizar un `Locator` también son transparentes al programador. Lo único que necesitamos para un uso básico es ajustar la configuración. No hay que tocar ni una sola línea de código respecto a nuestro servidor «hola mundo» que creaba proxies directos.

Para nuestro servidor *Printer* simplemente necesitamos un fichero como el siguiente:

Listado 2.2: Configuración del servidor *Printer* para uso de un `Locator`.
locator/server.config

```
1 PrinterAdapter.Endpoints=tcp
2 PrinterAdapter.AdapterId=PrinterAdapter1
3 PrinterAdapter.Locator=IceGrid/Locator -t:tcp -h 127.0.0.1 -p 4061
```

La **línea 1** es equivalente al fichero de configuración que ya usamos en el «hola mundo», salvo que en esta ocasión no incluye el número de puerto, de modo que el *runtime* lo elegirá arbitrariamente.

La **línea 2** es obligatoria y sirve para elegir un identificador único para el adaptador de objetos `PrinterAdapter` dentro del `Locator`. Por último, la **línea 3** fija para nuestro adaptador el proxy del `Locator` que es proporcionado por el programa que ejecutamos en la sección anterior.

En el directorio `locator` hay una copia del servidor y del cliente Python del «hola mundo» (`py/hello`). Para lanzar el servidor ejecuta el siguiente comando. Por supuesto, debe funcionar exactamente igual con los servidores de otros lenguajes si les indicas este mismo fichero de configuración.

```
locator$ ./server.py --Ice.Config=server.config
printer1 -t -e 1.1 @ PrinterAdapter1
```

Como se puede comprobar, el mismo servidor (idéntico código) ha generado un proxy indirecto, por el mero hecho de haber configurado un `Locator`.

2.3 Invocando un proxy indirecto

Tampoco es necesario modificar el código del cliente para poder utilizar un proxy indirecto, pero se le debe indicar el proxy del `Locator` para que pueda resolver los proxies indirectos. Para ello sirve el siguiente fichero de configuración:

Listado 2.3: Configuración del cliente para poder resolver proxies indirectos.
locator/locator.config

```
Ice.Default.Locator=IceGrid/Locator -t:tcp -h 127.0.0.1 -p 4061
```

Para ejecutar el cliente, debemos indicarle el fichero de configuración anterior y el proxy indirecto que imprimió el servidor en su salida.

```
locator$ ./client.py --Ice.Config=locator.config "printer1 -t @ PrinterAdapter1"
```

Al ejecutar este comando, la salida del servidor mostrará el mensaje «Hello World!» corroborando que todo funciona como se esperaba.

2.4 ¿Qué ocurre realmente?

¿Cómo puede el mismo cliente (sin cambiar nada en el código) encontrar el servidor y enviar el mensaje de invocación al nodo y puerto correctos?

Lógicamente ocurre porque el *runtime* de Ice modifica su comportamiento al cargar la nueva configuración. Este proceso se denomina *binding* y muy simplificado es algo así:

1. Cuando el servidor crea su adaptador de objetos y comprueba que hay un `Locator` disponible, le envía un mensaje al `Registry` asociado que incluye el `AdapterId` y los *endpoints* en los que va a escuchar el servidor.
2. Como el adaptador de objetos está registrado, cuando se añade el sirviente, el adaptador crea por defecto un proxy indirecto para el objeto.
3. Cuando el cliente trata de utilizar el proxy para enviar un mensaje y comprueba que es indirecto, utiliza el `Locator` disponible para obtener el correspondiente proxy directo.
4. Una vez dispone del proxy directo, puede construir un mensaje de invocación y enviarlo de la forma habitual.

Este proceso es transparente para el programador y ocurre la primera vez que se invoca a un objeto. Las invocaciones sucesivas no lo necesitan porque el *runtime* del cliente cachea los proxies ya resuelto. Si al utilizar un proxy cacheado ocurriera un fallo porque el servidor ya no está en ese lugar, el cliente trataría de conseguir el nuevo proxy directo (si es que lo hay) todo de forma transparente.

2.5 Ventajas

Como ya se ha dicho anteriormente, gracias a los proxies indirectos conseguimos *transparencia de localización*, que en la práctica implica que la referencia que usa el cliente:

- Es la misma aunque el nodo servidor cambie de dirección IP.
- Es la misma aunque el servidor arranque en otro puerto.
- Es la misma aunque una nueva versión del servidor disponga de nuevos *endpoints* y el cliente podrá usarlos.
- Es la misma aunque el servidor pare y arranque en otro nodo.

Para ilustrar este último punto, pruebe el cliente `py/client-loop.py` que repite la misma invocación cada segundo. Para y vuelve a arrancar el servidor mientras el cliente sigue en funcionamiento. Puedes comprobar cómo el cliente puede invocar la nueva instancia tan pronto esté disponible. Y funciona del mismo modo aunque la nueva de instancia del servidor la arranque en un nodo distinto.

Chapter 3

Gestión de aplicaciones distribuidas

[Python]

En este capítulo veremos el uso básico de `IceGrid`, un servicio incluido en ZeroC Ice que facilita la gestión de aplicaciones distribuidas en grid y proporciona mecanismos de redundancia y tolerancia a fallos entre otros. Además veremos cómo desplegar el software de la aplicación en cada nodo. Juntos permiten aplicar procedimientos avanzados sin necesidad de modificar ni una sola línea del código de la aplicación.

3.1 Introducción

Cuando cualquier sistema crece, inmediatamente aparecen problemas de gestión. Necesitamos saber qué servicio funciona y cuál falla, cuál está activo, cuál ha caído o está inaccesible. Necesitamos mecanismos para distribuir el código de cada servidor en el nodo que le corresponde, arrancarlo o pararlo y diagnosticar problemas que puedan estar ocurriendo.

En este capítulo utilizaremos exactamente el mismo programa del capítulo 1 y veremos cómo desplegar, arrancar y gestionar la aplicación (formada por un cliente y dos servidores) de un modo mucho más sistemático y escalable. Todo ello es posible en buena medida gracias a los proxies indirectos que vimos en el capítulo 2.

3.2 IceGrid

Ice incluye un conjunto de servicios cuidadosamente diseñados que cooperan para proporcionar una gran variedad de prestaciones avanzadas. Uno de los servicios más importantes es IceGrid.

IceGrid proporciona una gran variedad de funcionalidades para gestión remota de la aplicación distribuida, activación automática (implícita) de objetos, balanceo de carga y transparencia de localización.

3.2.1 Componentes de IceGrid

IceGrid depende de una base de datos compartida llamada *IceGrid Registry*. Contiene información sobre los objetos remotos conocidos por el sistema distribuido, las aplicaciones actualmente registradas, los nodos de cómputo disponibles y algunos otros datos. El *Registry* es un componente clave en el sistema y sólo puede existir una instancia, aunque puede estar replicado. Se puede decir que el Registry es el que determina qué objetos forman parte de la aplicación. Puede haber varios Registry en ejecución en el mismo grid, pero corresponderían a sistemas aislados con sus propias aplicaciones distribuidas, incluso aunque usen los mismos nodos.

Además, IceGrid requiere que cada nodo de cómputo asociado al sistema ejecute un servicio llamado *IceGrid Node*. Ésta es la forma en la que IceGrid puede determinar qué nodos de cómputo están disponibles para la ejecución de componentes de la aplicación.

Por último, IceGrid incluye un par de herramientas de administración que se pueden utilizar como interfaz con el usuario para controlar todas sus características. Existe una aplicación en línea de comando llamada `icegridadmin` y otra con interfaz gráfica llamada `icegridgui`. Usaremos la segunda en este capítulo.

IceGrid maneja algunos conceptos que es importante aclarar, debido a que no corresponden exactamente con el significado habitual:

Nodo
: Corresponde con una instancia de *IceGrid Node*. Identifica un «nodo lógico», es decir, no tiene por qué corresponder unívocamente con un computador, pudiendo haber más de un *IceGrid Node* ejecutándose sobre un mismo nodo de cómputo, que a su vez pueden estar vinculados al mismo Registry o a distintos.

Servidor
: Identifica, mediante un nombre único, a un programa que se ejecutará en un nodo. Incluye los atributos y propiedades que puedan ser necesarios para su configuración. Nótese que el programa a ejecutar puede ser también un cliente o incluso un programa que no tiene nada que ver con Ice.

Adaptador de objetos
: Incluye datos específicos de un adaptador de objetos utilizado en un servidor Ice, incluyendo endpoints, *objetos bien conocidos*, etc.

Aplicación
: Se refiere al conjunto de servicios, objetos y sus respectivas configuraciones que conforman la *aplicación distribuida*. Las descripciones de aplicaciones se pueden almacenar en la base de datos de IceGrid y también se pueden exportar como ficheros XML.

3.2.2 Configuración de IceGrid

Lo primero es configurar un conjunto de nodos que utilizaremos durante la etapa de desarrollo de la aplicación. Como mínimo necesitamos un *IceGrid Registry* y un *IceGrid node*, aunque pueden ejecutarse en un mismo computador, de hecho, en un mismo programa. Concretamente vamos a utilizar dos nodos IceGrid, y uno de ellos ejecutará también el Registry.

La configuración de IceGrid es muy similar a la de cualquier otra aplicación de Ice. Debemos proporcionar un conjunto de propiedades en forma de pares «clave=valor» en un fichero de texto plano (ver § 1.6).

Una de las propiedades que siempre debe tener la configuración de cualquier nodo IceGrid es el proxy al servicio *Locator*. El *Locator* lo proporciona la instancia del Registry y tiene la estructura que se muestra en el siguiente listado:

Listado 3.1: Configuración del nodo 1 (fragmento)
py/icegrid/node1.config

```
Ice.Default.Locator=IceGrid/Locator -t:tcp -h 127.0.0.1 -p 4061
```

La dirección IP 127.0.0.1 tendría que ser reemplazada por el nombre o la dirección IP del nodo que ejecuta el Registry. En este ejemplo mínimo, todos los componentes estarán en el mismo computador, por ese motivo, usamos esta dirección.



Recuerda que la dirección IP 127.0.0.1 es una dirección especial que se asigna a la interfaz *loopback*. Permite comunicar servidores y clientes que se ejecutan en el mismo computador sin necesidad de una NIC. Todo computador que ejecute la pila de protocolos TCP/IP tiene esta interfaz con esa dirección.

Cada nodo debe proporcionar un nombre identificativo y un directorio en el que almacenar la base de datos del nodo. El programa `icegridnode` es una aplicación Ice convencional, como nuestro servidor de «hola mundo». Por tanto, se deben proporcionar los *endpoints* para el adaptador de objetos del nodo IceGrid.

Listado 3.2: Configuración del nodo 1 (continuación)
py/icegrid/node1.config

```
1 IceGrid.Node.Name=node1
2 IceGrid.Node.Data=/tmp/icedata/node1
3 IceGrid.Node.Endpoints=tcp
```

Eso es todo lo que se necesita para configurar un nodo IceGrid normal. Pero necesitamos un nodo configurado especialmente para ejecutar también un Registry. Para ello utilizamos la siguiente propiedad:

Listado 3.3: Configuración del Registry (continuación)
py/icegrid/node1.config

```
IceGrid.Node.CollocateRegistry=1
```

IANA

La IANA estandariza los puertos TCP y UDP para servicios comunes.

Cuando esta propiedad tiene un valor distinto de cero, una instancia de IceGrid Registry se añade al adaptador que se especifique (mediante propiedades específicas). El nodo tiene tres adaptadores diferentes dado que se usan para propósitos distintos con privilegios específicos. El adaptador «Client» se utiliza para registrar el servicio Locator, por lo que debería estar configurado para usar el puerto estándar 4061. También debemos indicar un directorio para la base de datos del Registry (**línea 4**).

Listado 3.4: Configuración del Registry (continuación)
py/icegrid/node1.config

```
1 IceGrid.Registry.Client.Endpoints=tcp -p 4061:ws -p 4065
2 IceGrid.Registry.Server.Endpoints=tcp
3 IceGrid.Registry.Internal.Endpoints=tcp
4 IceGrid.Registry.LMDB.Path=/tmp/icedata/registry
```

El acceso al Registry está controlado por un verificador de permisos —un servicio simple para autenticación y autorización de usuarios. Existe una implementación hueca (*dummy*) que se suele utilizar durante el desarrollo de la aplicación:

Listado 3.5: Configuración del Registry (continuación)
py/icegrid/node1.config

```
IceGrid.Registry.PermissionsVerifier=IceGrid/NullPermissionsVerifier
IceGrid.Registry.AdminPermissionsVerifier=IceGrid/NullPermissionsVerifier
```

Como se mencionó anteriormente, utilizaremos dos nodos. Por eso necesitamos proporcionar otro fichero de configuración. En este caso sólo se necesitan las propiedades esenciales: El proxy al `Locator`, el nombre del nodo, el directorio de la base de datos y los endpoints del adaptador del nodo:

Listado 3.6: Configuración de un nodo IceGrid convencional (sin Registry)
py/icegrid/node2.config

```
Ice.Default.Locator=IceGrid/Locator -t:tcp -h 127.0.0.1 -p 4061
IceGrid.Node.Name=node2
IceGrid.Node.Data=/tmp/icedata/node2
IceGrid.Node.Endpoints=tcp
```

Recuerda que si estás ejecutando los dos nodos en computadores diferentes, debes cambiar la dirección IP por la dirección que tenga el computador en la que se ejecuta el Registry.

Se requiere un fichero de configuración adicional para los clientes que deseen utilizar el `Locator`:

Listado 3.7: Configuración del Locator
py/icegrid/locator.config

```
Ice.Default.Locator=IceGrid/Locator -t:tcp -h 127.0.0.1 -p 4061
```

3.2.3 Arrancando IceGrid

El nodo IceGrid normalmente se ejecuta como un *servicio* del sistema (coloquialmente llamados *demonios*) que arranca automáticamente al conectar cada computador asociado al sistema distribuido y que se ejecuta en *background*. En este caso y con propósitos didácticos vamos a proceder a arrancarlos manualmente y en primer plano. Por tanto, si tu instalación de Ice los ha arrancado como *demonios* debes pararlos antes de continuar.

Abre un terminal en el directorio `py/icegrid`, crea los directorios para las bases de datos del nodo y el registro y lanza `icegridnode` con la configuración del *node 1*:

```
py/icegrid$ mkdir -p /tmp/icedata/node1 /tmp/icedata/registry
py/icegrid$ icegridnode --Ice.Config=node1.config
```

Ese terminal queda ocupado con la ejecución del nodo. Déjalo así, en la salida puede imprimir información útil si algo va mal. Cuando acabemos podrás pulsar Control-C para pararlo.

Abre un segundo terminal en el mismo directorio, crea el directorio para la base de datos y ejecuta otra instancia de `icegridnode`, esta vez con la configuración del nodo 2:

```
py/icegrid$ mkdir -p /tmp/icedata/node2
py/icegrid$ icegridnode --Ice.Config=node2.config
```

Si todo funciona correctamente, tendrá el mismo efecto y el terminal quedará ocupado.

Ahora procedemos a conectar con el Registry utilizando la aplicación `icegridgui`, que puedes lanzar en un tercer terminal:

```
$ icegridgui
```

Deberías ver una ventana similar a la de la figura 3.1.

3.3 Creando una aplicación distribuida

A continuación crearemos una nueva aplicación con `icegridgui`. Pero primero necesitamos conectar con el Registry pulsando el botón *Log into an IceGrid Registry* situado arriba a la izquierda, el primero de la barra de herramientas ().

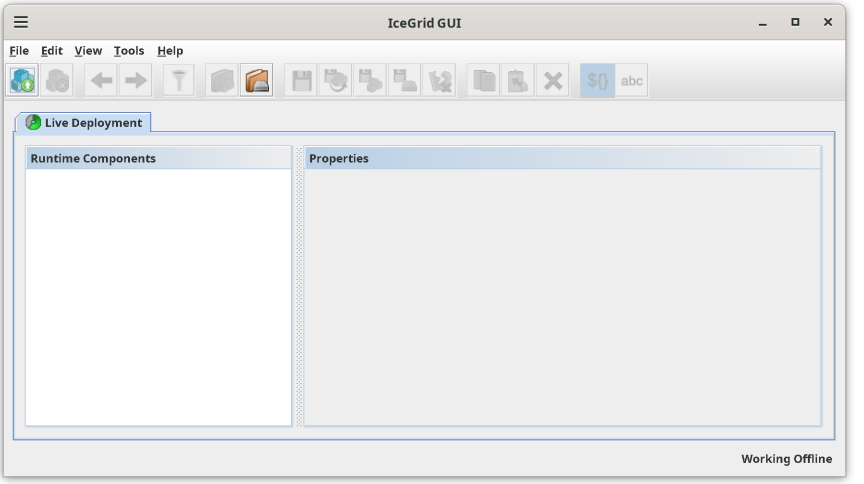


Figure 3.1: IceGrid Admin: Ventana inicial

Aparecerá una ventana con una lista de conexiones como la de la figura 3.2. Pulsa New connection, lo que inicia un pequeño asistente para crear una nueva conexión. Ese asistente tiene los siguientes pasos:

1. Tipo de conexión (figura 3.3). Elige *Direct connection*, pulsa Next.
2. Marcar *Connect to a Master Registry*, pulsa Next.
3. Elegir el Registry descubierto automáticamente, pulsa Next.
4. Aceptar los endpoints descubiertos, pulsa Next.
5. Credenciales (figura 3.4). Escribe «user» y «pass» y pulsa Finish. Por supuesto, en una aplicación en producción, se requeriría una contraseña o un mecanismo de autenticación equivalente. Recuerda que el registro está configurado con un verificador de permisos *dummy* (falso).



Figure 3.2: IceGrid Admin: Conexiones guardadas

Cuando la aplicación ha establecido el acceso al registro debería aparecer una ventana similar a la de la figura 3.6. Esta ventana muestra una vista lógica actualizada del sistema distribuido. Puedes ver todos los nodos involucrados en el



Figure 3.3: IceGrid Admin: Nueva conexión



Figure 3.4: IceGrid Admin: Credenciales

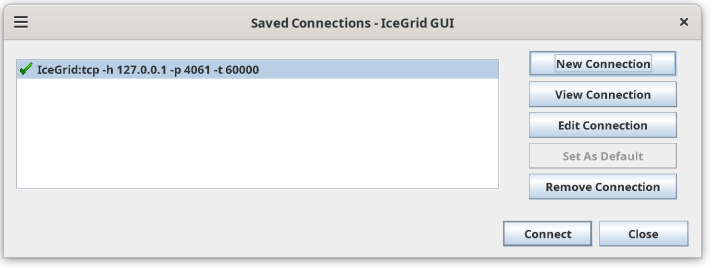


Figure 3.5: IceGrid Admin: Conexión creada

sistema, es decir, los que están configurados con el Locator asociado al Registry al que has conectado. Puedes pulsar los iconos que representan esos nodos y obtener datos actualizados con información sobre su sistema operativo, su carga, etc.

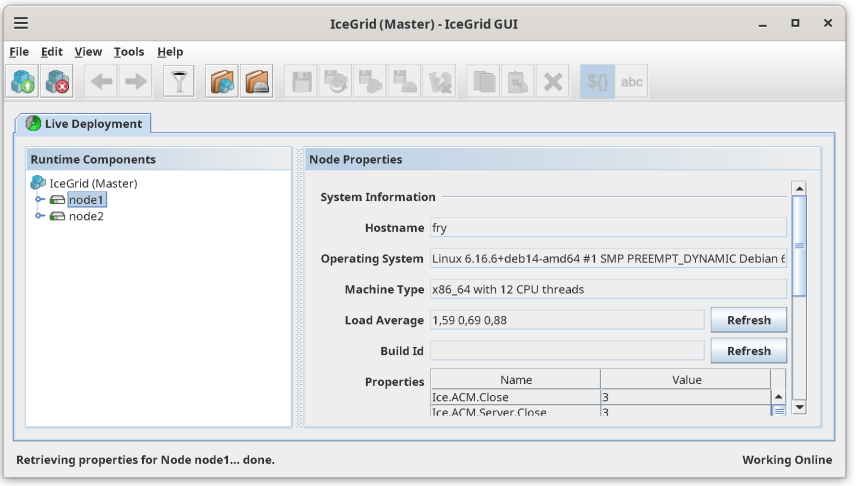


Figure 3.6: IceGrid Admin: Ventana *Live Deployment*

Para crear una nueva aplicación selecciona la opción de menú *File, New, Application* (figura 3.7). Aparecerá una nueva pestaña llamada *NewApplication*. Edita el nombre de la aplicación, escribe «PrinterApp» y pulsa Apply.

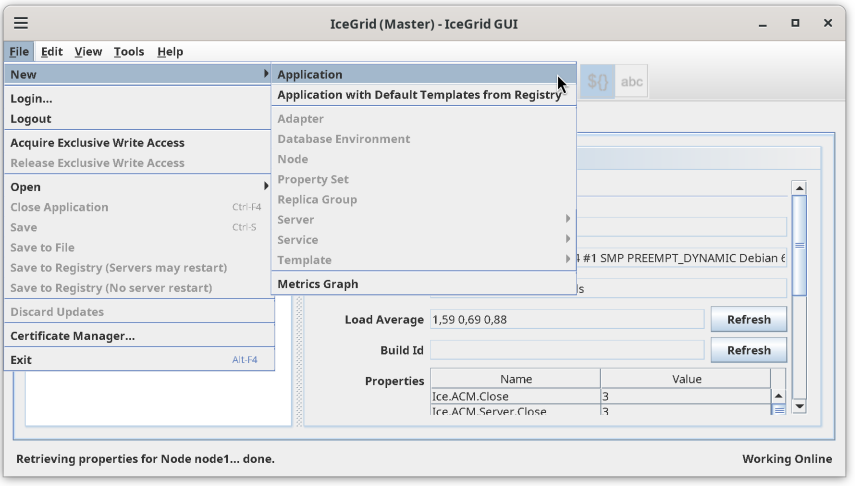


Figure 3.7: IceGrid Admin: Creación de aplicaciones

Abre el menú contextual (botón derecho del ratón) de la carpeta *Nodes* situado en el panel izquierdo (figura 3.8). Crea un nuevo nodo, cambia su nombre a `node1` y pulsa Apply. Crea otro nodo, nómbralo como `node2` y pulsa Apply de nuevo. Ahora

debería haber dos nodos con los mismos nombres que los que aparecen en la pestaña *Live Deployment*. Comprueba que efectivamente los nombres corresponden (respetando mayúsculas y minúsculas).

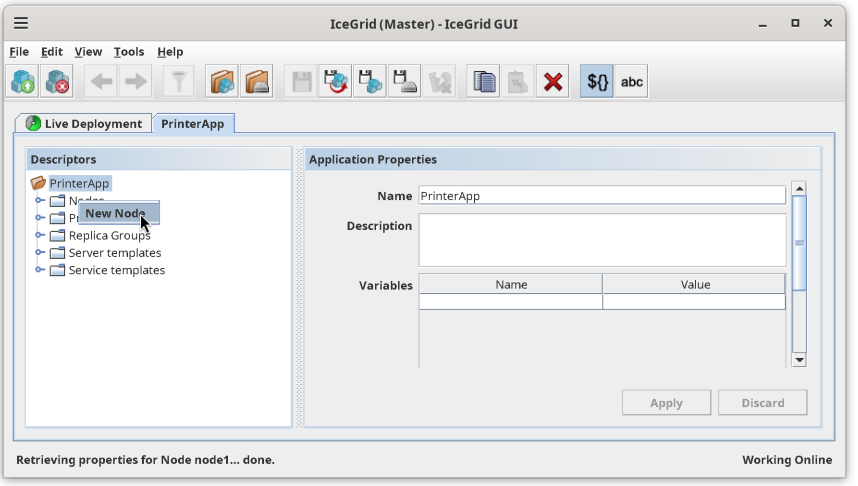


Figure 3.8: IceGrid Admin: Creación de un nodo

3.4 Instanciación del servidor

Es hora de crear el descriptor para el servidor de «hola mundo»:

- En el menú contextual del nodo `node1` elige *New Server* y cambia el nombre por defecto a `PrinterServer1`.
- Edita la propiedad *Path to Executable* y escribe `./server.py`
- En la propiedad *Working Directory* escribe `/tmp/printer-py/${node}`.

IceGrid se encarga de arrancar y parar los servidores, pero no de copiar sus programas en los nodos. El descriptor solo indica qué programa ejecutar y en qué directorio, y es tarea nuestra que esos ficheros estén ahí para el momento en que deba arranca el servidor (ver § 3.5.1). La variable `${node}` se sustituye por el nombre del nodo en el que se ejecuta el servidor. En un despliegue real cada nodo estaría en un computador distinto y bastaría con un directorio fijo, pero en este ejemplo todos los nodos comparten computador y así cada uno tiene su propia copia de los programas.

Tal como vimos en la sección 1.6, el adaptador de objetos del servidor requiere configuración adicional, en concreto, debemos proporcionarle los *endpoints* en los que escuchar. Sin embargo, en este caso la configuración de ese adaptador la fijaremos a través de IceGrid.

En el menú contextual de `PrinterServer1` elige *New Adapter*. Cambia el nombre por defecto a `PrinterAdapter` y pulsa Apply. Este nombre debe corresponder con

el nombre del adaptador definido en el código del servidor mediante el método `createObjectAdapter()`.

Sin embargo, tenemos un problema: ¿Cómo sabremos si el servidor funciona correctamente? Su única función es imprimir en su salida estándar las cadenas que se le envían. Para poder obtener remotamente las salidas estándar y de error del servidor configuramos dos propiedades adicionales.

- Pulsa sobre `PrinterServer1` en el lado izquierdo.
- Pulsa en la tabla `Properties` y añade una propiedad llamada `Ice.StdOut`.
- Como valor escribe `/tmp/printer-py/${node}/server-out.txt`.
- Añade otra propiedad llamada `Ice.StdErr` con el valor `/tmp/printer-py/${node}/server-err.txt` y pulsa Apply.

Debería quedar como la figura 3.9.

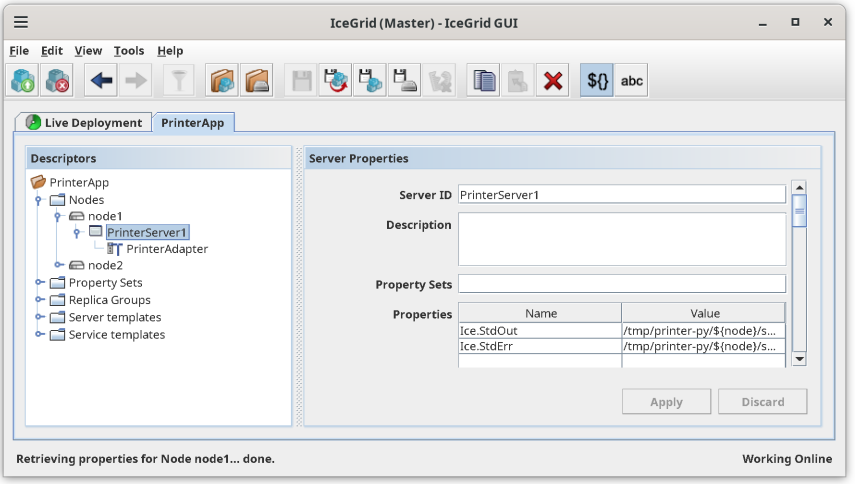


Figure 3.9: IceGrid Admin: Instancia de la aplicación servidor

3.4.1 Instanciando un segundo servidor

Crearemos ahora una segunda instancia del servidor de impresión en otro nodo. Sigue los mismos pasos que en la sección anterior con estas diferencias:

- Crea el servidor en el nodo `node2` y llámalo `PrinterServer2`.
- En la propiedad *Path to Executable* escribe `./server-uuid.py`

La única diferencia es que este servidor contendrá un objeto cuya identidad se genera aleatoriamente (ver método `addWithUUID()` del adaptador de objetos).

3.5 Ejecutando la aplicación

Para empezar, guarda la aplicación recién definida en el Registry pulsando el botón *Save to registry* ().

3.5.1 Despliegue de la aplicación

El «despliegue» (en inglés *deployment*) es el proceso de instalación del software de la aplicación en los diferentes nodos que forman el sistema, es decir, transferir a cada nodo los programas (servidores, en el caso de Ice) y los ficheros que necesiten para hacer su trabajo. Dado que la aplicación puede ejecutarse sobre un grid con muchos nodos, no es razonable hacerlo a mano.

Para eso utilizaremos `Ansible`, una herramienta de automatización que permite ejecutar tareas en remoto sobre un conjunto de nodos. `Ansible` se conecta a los nodos mediante SSH y ejecuta en ellos las tareas descritas en un *playbook*. No requiere instalar ningún servicio en los nodos, basta con que tengan un intérprete de Python.

Los nodos sobre los que trabaja Ansible se indican en un fichero llamado *inventario*. En nuestro caso hay computador por cada nodo IceGrid, con el mismo nombre:

Listado 3.8: Inventario de Ansible
py/icegrid/inventory.ini

```
[nodes]
node1
node2

[all:vars]
ansible_connection=local
ansible_python_interpreter=/usr/bin/python3
```

Como en este ejemplo ambos nodos IceGrid se ejecutan en el mismo computador, la variable `ansible_connection=local` indica a Ansible que no necesita conectarse por SSH. En un despliegue real habría que indicar la dirección de cada computador con la variable `ansible_host`.

Ansible no compila los programas, solo copia lo que encuentra en el subdirectorio `dist` del ejemplo. Por eso, antes de desplegar hay que preparar la «distribución», es decir, los ficheros concretos que se pretenden desplegar. Para ello basta ejecutar `make gen-dist` en el directorio del ejemplo (`py/hello`), que copia en `dist` los ficheros necesarios.

El *playbook* `deploy.yml` realiza el despliegue en tres pasos:

1. Deshabilita los servidores activos. Es necesario deshabilitarlos para evitar que IceGrid los arranque mientras se copian los ficheros. Estas tareas se ejecutan en el nodo destino con `icegridadmin`.

2. Copia el contenido de `dist` en el directorio `/tmp/printer-py/`*nodo* de cada nodo (Listado 3.9), que es precisamente el que indicamos como directorio de trabajo de los servidores.

3. Vuelve a habilitar los servidores.

Listado 3.9: Copia de los programas en los nodos (fragmento)
py/icegrid/deploy.yml

```
- name: Copy the programs to the nodes
  hosts: nodes
  vars:
    src: "{{ playbook_dir }}/../hello/dist/"
    dest: "/tmp/printer-py/{{ inventory_hostname }}"
  tasks:
    - name: Create the destination directory
      ansible.builtin.file:
        path: "{{ dest }}"
        state: directory

    - name: Copy the distribution
      ansible.builtin.copy:
        src: "{{ src }}"
        dest: "{{ dest }}/"
        mode: preserve
```

Los nombres de los servidores están en `py/icegrid/group_vars/all.yml`. Para hacer el despliegue, ejecuta en el directorio `py/icegrid`:

```
py/icegrid$ make -C ../hello gen-dist
py/icegrid$ ansible-playbook -i inventory.ini deploy.yml
```

Si todo ha ido bien, el resumen final (*PLAY RECAP*) no mostrará ninguna tarea fallida (`failed=0`) y en cada nodo habrá una copia de los programas.

3.5.2 Ejecutando los servidores

Haz doble click en `node1` en la pestaña *Live Deployment*. Aparecerá su «contenido», concretamente el servidor `PrinterServer1`. Abre el menú contextual de `PrinterServer1` y pulsa *Start*. Un símbolo de «play» aparecerá sobre él y la barra de estado mostrará el mensaje «Starting server 'PrinterServer1'.. done».

Ahora que el servidor está en marcha puedes ver su salida estándar. Sólo tienes que abrir de nuevo el menú contextual de `PrinterServer1` y pulsar *Retrieve stdout*. Aparecerá una ventana similar a la de la figura 3.10.

Como recordarás, el servidor de impresión escribe en consola la representación textual del proxy de su objeto (ver § 1.6). Sin embargo, la salida que ahora muestra el servidor es un proxy indirecto, dado que cuando un servidor se ejecuta dentro de un nodo IceGrid tiene definido un `Locator` y se comporta de forma similar a como vimos en el capítulo 2.

```
printer1 -t -e 1.1 @ PrinterServer1.PrinterAdapter
```

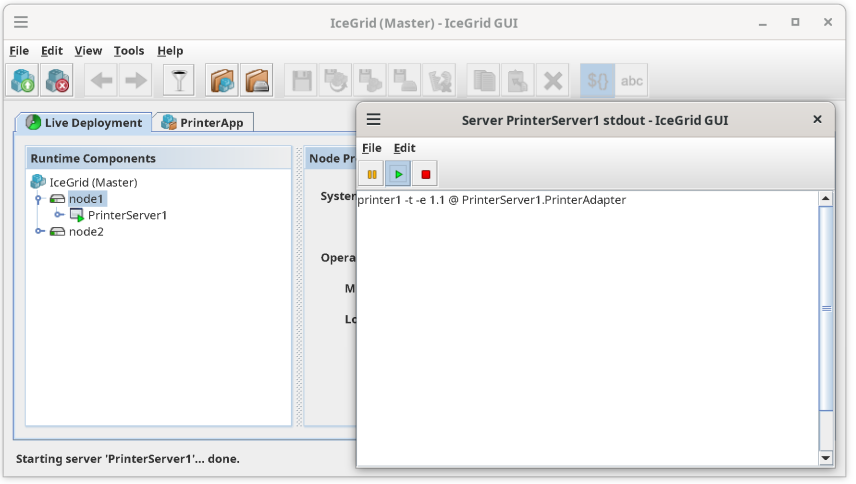


FIGURE 3.10: IceGrid Admin: Salida estándar del servidor

Recuerda que a partir de un proxy indirecto, el cliente debe pedir al servicio Locator los detalles de conexión que correspondan, convirtiendo así un proxy indirecto en uno directo. El Locator buscará el adaptador `PrinterServer1.PrinterAdapter` y reemplazará el `@PrinterServer1.PrinterAdapter` por los endpoints de dicho adaptador. El adaptador debe ser el único con ese identificador público. Para entender la diferencia con el nombre del adaptador de objetos pulsa otra vez en la aplicación `PrinterApp` y después en el adaptador `PrinterAdapter` dentro de `PrinterServer1` (ver figura 3.11).

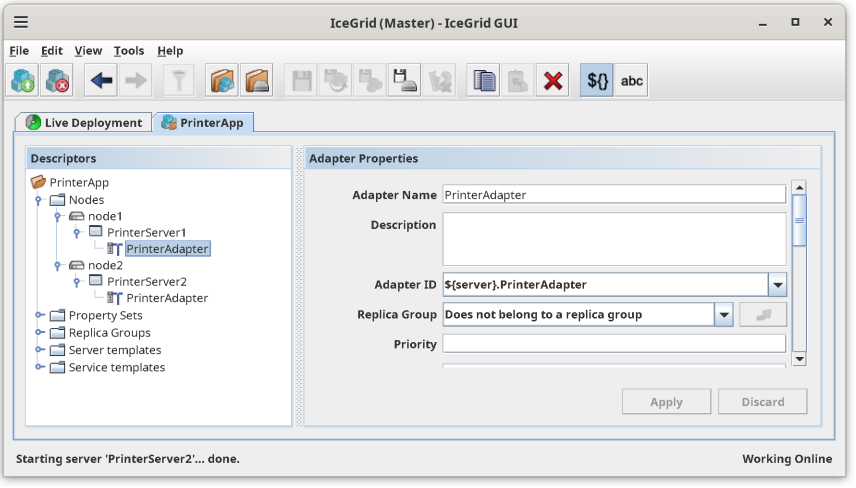


FIGURE 3.11: IceGrid Admin: Configuración del adaptador de objetos



Nótese que el *Adapter ID* tiene un valor por defecto igual a `${server}.PrinterAdapter`. La variable `${server}` se traducirá por el nombre del servidor (`PrinterServer1` en este caso) y el resto corresponde con el nombre del adaptador de objetos. Puede parecer raro que tenga que existir un nombre interno y un identificador público, pero esto permite crear instancias diferentes del mismo servidor sin que se repita el identificador del adaptador.

Para ejecutar el segundo servidor, procede del mismo modo desplegando el `node2` y pulsa *Start* en el `PrinterServer2`.

3.5.3 Ejecutando el cliente

Recuerda que el cliente acepta un argumento desde la línea de comandos: la representación textual del proxy que debe utilizar. Ahora conocemos el valor de ese parámetro, así que ya podemos ejecutar el cliente:

```
py/hello$ ./client.py --Ice.Config=../icegrid/locator.config \
        "printer1 -t -e 1.1 @ PrinterServer1.PrinterAdapter"
```

Nótese el parámetro `--Ice.Config` en el que se le indica al cliente el fichero en el que aparece el valor de la propiedad *Ice.Default.Locator*. Sin esa referencia, el cliente no sabría cómo resolver proxies indirectos.

Mira la salida estándar del `PrinterServer1` (figura 3.12). ¡Funciona!

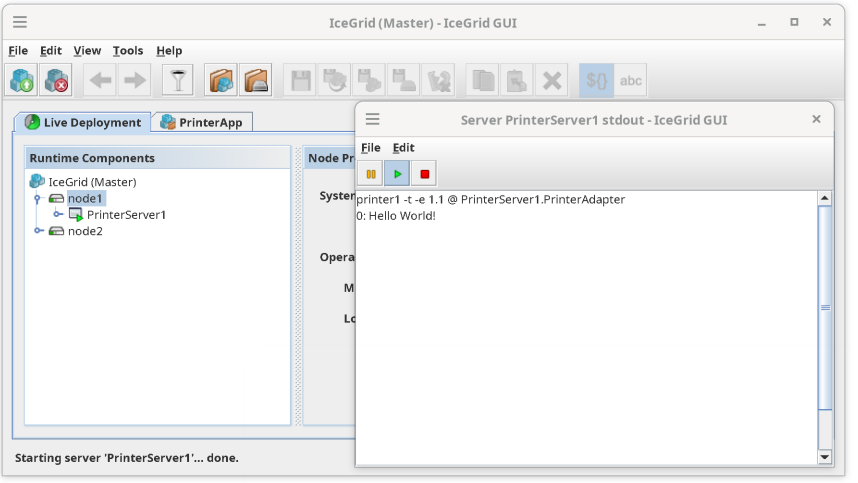


Figure 3.12: IceGrid Admin: Salida estándar del servidor después de ejecutar el cliente

Sólo tenemos que usar `printer1@PrinterServer1.PrinterAdapter`. A diferencia del proxy directo, esta designación siempre será la misma, aunque el servidor se ejecute en **un nodo diferente** o el middleware elija un puerto distinto. Además, la

indirección no resulta especialmente ineficiente puesto que el cliente solo preguntará al Locator la primera vez y cacheará la respuesta. Sólo volverá a consultar al Locator si falla una invocación sobre el proxy cacheado, y todo esto de forma automática y sin la intervención del programador.

3.6 Objetos bien conocidos

Algunos objetos importantes para la aplicación pueden ser incluso más fáciles de localizar. Los objetos bien conocidos pueden ser localizados simplemente por sus nombres. Es bastante sencillo conseguirlo. Ve a la pestaña `PrinterApp` y luego al adaptador `PrinterAdapter` de `PrinterServer1`. Desplaza la ventana hacia abajo hasta la tabla *Well-known objects*. Añade la identidad `printer1` y pulsa Apply, tal como se muestra en la figura 3.13.

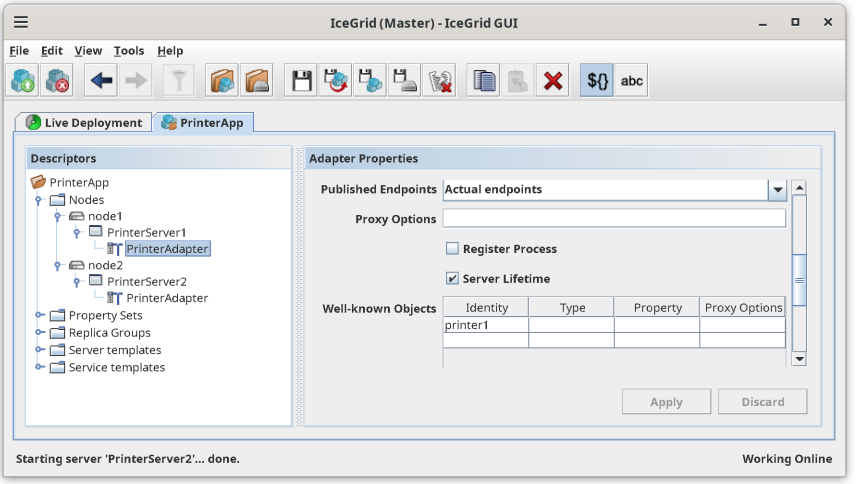


Figure 3.13: IceGrid Admin: Añadiendo un objeto bien conocido

Para invocar el objeto bien conocido con el cliente simplemente debes indicar el nombre como proxy textual:

```
py/hello$ ./client.py --Ice.Config=../icegrid/locator.config printer1
```

Mira la salida estándar de `PrinterServer1` y comprueba que también funciona. Una cuestión interesante es que ni siquiera ha sido necesario reiniciar el servidor. En este caso, la cadena `printer1` es otra forma de proxy indirecto. La notación `identidad@adaptador` se refiere a un objeto ligado a un adaptador de objetos registrado (aquel que tiene un *Adapter ID*), mientras que la notación `identidad` corresponde con objetos *bien conocidos*.

3.7 Activación y desactivación implícita

Sería ideal si los servidores solo estuvieran en ejecución cuando realmente son necesarios en lugar de consumir recursos todo el tiempo. Los proxies indirectos son ideales para conseguirlo de forma automática. Si un cliente intenta contactar con un proxy indirecto por primera vez, hará una petición al Locator. Si falla, intentará contactar al Locator de nuevo.

El Registry es capaz de usar las peticiones al Locator como solicitudes de activación implícita. Pulsa en la pestaña *PrinterApp* y luego en *PrinterServer1*. Despliega el menú *Activation mode* y selecciona *on-demand*. Ahora pulsa en Apply y en *Save to registry*.

Ahora ve a la pestaña *Live Deployment*, en el menú contextual de `PrinterServer1` pulsa *Stop*. Ahora vuelve a ejecutar el cliente invocando a `printer1`. El servidor `PrinterServer1` arrancará automáticamente en el momento en el que el cliente intenta conectar con él.

También es bastante conveniente desactivar los servicios ociosos pasado un tiempo de inactividad. Para hacerlo posible ve a la pestaña `PrinterApp`, pulsa en `PrinterServer1` y en la tabla de propiedades añade una nueva propiedad llamada `Ice.ServerIdleTime` con el valor '`5`'. Pulsa Apply y luego el botón *Save to registry*. Después de 5 segundos de inactividad, el servidor se apagará automáticamente.

3.8 Depuración

Es fácil cometer algún error durante la configuración de la aplicación distribuida y puede resultar complicado si no se es sistemático al tratar de identificar el problema.

Si algo va mal primero deberías revisar la salida de error estándar del programa que falla. Para que esta salida esté disponible desde la herramienta de administración se debe definir la propiedad `Ice.StdErr` y darle una ruta a un fichero tal que `/tmp/printer-py/${node}/server-err.txt`. Una vez que el programa se ejecute podrás acceder a su contenido por medio del menú contextual del servidor en la opción *Retrieve stderr*. Del mismo modo se puede activar la salida estándar con la propiedad `Ice.StdOut`. Estas mismas propiedades pueden definirse en la configuración de los nodos por medio de los ficheros que aparecen en la sección 3.2.2. Esto nos dará información extra muy útil para localizar y resolver posibles problemas.

3.8.1 Actualizando los servidores

Recuerda que si cambias un programa debes volver a generar la distribución y desplegarla (ver § 3.5.1). El *playbook* para los servidores que estén en marcha antes de copiar los ficheros, pero no los vuelve a arrancar. Tendrás que hacerlo tú, salvo que estén configurados con activación implícita.

3.8.2 Descripción de la aplicación

Recuerda que la aplicación queda definida en un fichero XML. Toda la configuración que se realiza en la definición de la aplicación en `icegridgui` queda almacenada en un fichero bastante legible/editable manualmente. Para nuestra aplicación ese fichero se muestra en el listado 3.10. Todo lo que hemos hecho en la segunda pestaña de `icegridgui` en realidad solo sirve para editar este fichero XML de una forma más visual.

LISTADO 3.10: Descripción de la aplicación PrinterApp en XML
py/icegrid/printerapp.xml

```
1       <node name="node1">
2          <server id="PrinterServer1" activation="manual" exe="./server.py" pwd="/tmp/printer
               -py/${node}">
3             <properties>
4                <property name="Ice.StdOut" value="/tmp/printer-py/${node}/server-out.txt"/>
5                <property name="Ice.StdErr" value="/tmp/printer-py/${node}/server-err.txt"/>
6             </properties>
7             <adapter name="PrinterAdapter" endpoints="default" id="${server}.PrinterAdapter"
                  />
8          </server>
9       </node>
10      <node name="node2">
11         <server id="PrinterServer2" activation="manual" exe="./server-uuid.py" pwd="/tmp/
               printer-py/${node}">
12            <properties>
13               <property name="Ice.StdOut" value="/tmp/printer-py/${node}/server-out.txt"/>
14               <property name="Ice.StdErr" value="/tmp/printer-py/${node}/server-err.txt"/>
15            </properties>
16            <adapter name="PrinterAdapter" endpoints="default" id="${server}.PrinterAdapter"
                  />
17         </server>
18      </node>
```

3.9 Receta

A continuación se listan a modo de resumen los pasos más importantes para realizar un despliegue cliente/servidor sencillo como el que se ha descrito:

- Crea un fichero de configuración para el `Registry` y para cada uno de los nodos del grid (§ 3.2.2).
- Arranca un `icegridnode` por cada nodo con su configuración correspondiente (§ 3.2.3).
- Crea la aplicación distribuida (§ 3.3).
- Crea un nodo en la aplicación para cada nodo ejecutado.
- Crea los servidores IceGrid (§ 3.4) en sus respectivos nodos, indicando el directorio en el que estarán sus programas.
- Guarda la aplicación en el Registry.
- Despliega los programas en los nodos con Ansible (§ 3.5.1).

- Arranca los servidores (§ 3.5.2).
- Ejecuta el cliente (§ 3.5.3).

3.10 Ejercicios

E 3.01 Invoca el servidor gestionado por `IceGrid` ejecutando el cliente desde otro servidor `IceGrid`.

E 3.02 Modifica el código del cliente para que invoque a los dos servidores.

E 3.03 El servidor del `node2` tiene una identidad aleatoria. Averigua cómo fijar la identidad del objeto por medio de una propiedad. Cambia el programa que ejecutan ambos servidores para ejecutar la nueva versión (ambos nodos el mismo programa) e indica por medio de una propiedad que sus identidades son respectivamente «printer1» y «printer2».

E 3.04 Define nodos adicionales, crea una plantilla[1] para el servidor `PrinterServer`. Arranca nodos adicionales y, usando la plantilla, instancia varias impresoras (objetos *Printer*) en ellos. Invoca esas impresoras con el cliente original desde la consola. Consulta el ejemplo `python/IceGrid/simple` de las *demos* de Ice[2].

E 3.05 Explora las posibilidades de `icegridadmin`[3], la herramienta de administración en línea de comandos, para obtener información y modificar la configuración de la aplicación. ¿Qué comando debes introducir para obtener la lista de nodos? ¿Y la lista de servidores?

E 3.06 Repite el tutorial de este capítulo usando un computador diferente para cada nodo. Esos computadores pueden ser virtuales (con `VirtualBox` por ejemplo) o contenedores docker. Tendrás que modificar el inventario de Ansible para indicar la dirección de cada uno.

[1] https://doc.zeroc.com/display/Ice/IceGrid+Templates
[2] https://github.com/zeroc-ice/ice-demos/tree/3.7
[3] https://doc.zeroc.com/display/Ice/icegridadmin+Command+Line+Tool

Chapter 4

Difusión de eventos

[Python]

Uno de los servicios más sencillos y útiles de Ice es IceStorm. Lo habitual en este tipo de servicios en otros middlewares, como NotificationService de CORBA, Data Distribution Service (DDS) o Java Message Service (JMS), es que los eventos que se envían sean objetos o estructuras con datos de estilo Data Transfer Object (DTO). Sin embargo, IceStorm propaga invocaciones a métodos, con lo que es prácticamente una implementación distribuida del patrón de diseño publicación/suscripción, más conocido como «observador» [GHJV94].

Para utilizar el servicio de eventos se requiere un *TopicManager*. Este objeto permite listar, crear y destruir canales (*topics*). Los canales también son objetos, que residen en el mismo servidor del *TopicManager*, dado que es él el que los crea.

Para recibir eventos procedentes de un canal es necesario suscribir un objeto, es decir, darle al canal la referencia (el proxy) del objeto. Para enviar invocaciones a un canal se necesita el proxy al *publicador del canal*. El publicador es un objeto especial que no tiene interfaz concreta. Se puede invocar cualquier método sobre él. El canal enviará esa invocación a todos sus suscriptores. Sin embargo, si el suscriptor no implementa la interfaz que corresponde con invocación, elevará una excepción y el canal lo des-suscribirá inmediatamente. Por esa razón se deben crear canales diferentes para cada interfaz de la que se quiera propagar eventos.

Los canales solo pueden propagar invocaciones oneway o datagram, es decir, que no tengan valores de retorno, dado que sería complejo tratar las respuestas de todos los suscriptores hacia un mismo origen. El canal es un intermediario que desacopla completamente a publicadores y suscriptores. Todos conocen el canal, pero no se conocen entre sí.

Veamos su funcionamiento con un ejemplo. Se crea un canal de eventos para la interfaz *Example.Printer* del capítulo anterior. Se empieza por codificar el suscriptor, que es en esencia un servidor. El código completo para el suscriptor se muestra el en siguiente listado:

Listado 4.1: Suscriptor Python
py/icestorm/subscriber.py

```
1  import sys
2  import Ice
3  import IceStorm
4  from pathlib import Path
5
6  Ice.loadSlice(str(Path(__file__).parent / 'printer.ice'))
7  import Example
8
9  class PrinterI(Example.Printer):
10     def write(self, message, current=None):
11         print(f"Event received: {message}")
12
13 def get_topic_manager(ic):
14     key = 'IceStorm.TopicManager.Proxy'
15     proxy = ic.propertyToProxy(key)
16     if proxy is None:
17         raise KeyError(f"property '{key}' not set")
18
19     print("Using IceStorm in: '%s'" % key)
20     retval = IceStorm.TopicManagerPrx.checkedCast(proxy)
21     if not retval:
22         raise ValueError("Invalid proxy for TopicManager")
23
24     return retval
25
26 def main(ic):
27     topic_mgr = get_topic_manager(ic)
28     servant = PrinterI()
29     adapter = ic.createObjectAdapter("PrinterAdapter")
30     subscriber = adapter.addWithUUID(servant)
31
32     topic_name = "PrinterTopic"
33     qos = {}
34     try:
35         topic = topic_mgr.retrieve(topic_name)
36     except IceStorm.NoSuchTopic:
37         topic = topic_mgr.create(topic_name)
38
39     topic.subscribeAndGetPublisher(qos, subscriber)
40     print("Waiting events... '{}'".format(subscriber))
41
42     adapter.activate()
43     ic.waitForShutdown()
44
45     topic.unsubscribe(subscriber)
46
47     return 0
```

Lo primero es conseguir el proxy al *TopicManager* (**línea 27**), que se obtiene a partir de una propiedad en el fichero de configuración (función `get_topic_manager()`).

A continuación se crea un sirviente y un adaptador de objetos, y se añade el sirviente al adaptador (**líneas 28–30**).

Después se obtiene una referencia al canal `PrinterTopic` (**línea 35**), que debería existir. Si no existe se produce una excepción (*NoSuchTopic*), que es capturada y su manejador crea el canal (**línea 37**). En cualquier caso, después de obtener el canal, se suscribe el objeto (**línea 39**). Por último, se activa el adaptador y el servidor queda a la espera de recibir eventos. Cuando el comunicador termina, se cancela la suscripción con `unsubscribe()`.

Si el suscriptor es como un servidor, el publicador es como un cliente. Solo se muestra la función `main()` puesto que el resto del código es prácticamente idéntico al suscriptor:

Listado 4.2: Publicador Python
py/icestorm/publisher.py

```
1  def main(ic):
2      topic_mgr = get_topic_manager(ic)
3
4      topic_name = "PrinterTopic"
5      try:
6          topic = topic_mgr.retrieve(topic_name)
7      except IceStorm.NoSuchTopic:
8          topic = topic_mgr.create(topic_name)
9
10     publisher = topic.getPublisher()
11     printer = Example.PrinterPrx.uncheckedCast(publisher)
12
13     print("publishing events")
14     for i in range(10):
15         printer.write(f"Hello World #{i}")
16         time.sleep(1)
17
18     return 0
```

Lo más interesante es la obtención del publicador del canal a partir de la referencia al canal (**línea 10**) y el *downcast* para poder invocar sobre él el método de la interfaz *Example::Printer* (**línea 11**). Nótese que este molde usa la modalidad `uncheckedCast()` dado que la comprobación fallaría puesto que el publicador no implementa realmente ninguna interfaz. Por último se utiliza el proxy `printer` para enviar un evento por segundo (hasta un total de 1000) con la cadena `"Hello World #n"`.

4.1 Arranque del servicio

IceStorm está implementado como un servicio Ice. Los servicios son librerías dinámicas que deben ser lanzadas con el servidor de aplicaciones, que se llama IceBox. Vemos la configuración de IceBox en el listado 4.3.

Listado 4.3: Configuración de IceBox para lanzar IceStorm
py/icestorm/icebox.config

```
IceBox.Service.IceStorm=IceStormService,37:createIceStorm --Ice.Config=icestorm.config
```

Sólo se definen dos propiedades: la carga del servicio IceStorm y los endpoints del gestor remoto de IceBox. La configuración de IceStorm a la que se hace referencia (`icestorm.config`) se muestra en el listado 4.4.

Listado 4.4: Configuración de IceStorm
py/icestorm/icestorm.config

```
IceStormAdmin.TopicManager.Default=IceStorm/TopicManager:tcp -p 10000
IceStorm.TopicManager.Endpoints=tcp -p 10000
IceStorm.Publish.Endpoints=tcp -p 2000
IceStorm.Flush.Timeout=2000
IceStorm.LMDB.Path=db
```

Las **líneas 1–3** especifican los endpoints para los adaptadores del *TopicManager*, el objeto de administración, y los publicadores de los canales. La **línea 5** indica el nombre del directorio donde se almacenará la configuración del servicio (que es persistente de forma automática).

Una vez configurado se puede lanzar el servicio y probar el ejemplo. Lo primero es crear ese directorio y arrancar IceBox (en *background*):

```
$ mkdir db
$ icebox --Ice.Config=icebox.config &
```

A continuación se puede usar la herramienta de administración de IceStorm en línea de comando para crear el canal, asumiendo que no existe:

```
$ icestormadmin --Ice.Config=icestorm.config -e "create PrinterTopic"
```

La configuración del suscriptor es:

Listado 4.5: Configuración del suscriptor
py/icestorm/subscriber.config

```
IceStorm.TopicManager.Proxy=IceStorm/TopicManager:tcp -p 10000
PrinterAdapter.Endpoints=tcp
```

Y con eso ya se puede arrancar el subscriptor:

```
$ ./subscriber.py --Ice.Config=subscriber.config
```

La configuración para el publicador es:

Listado 4.6: Configuración de publicador
py/icestorm/publisher.config

```
IceStorm.TopicManager.Proxy=IceStorm/TopicManager:tcp -p 10000
```

Y por último en una consola diferente se ejecuta el publicador:

```
$ ./publisher.py --Ice.Config=publisher.config
```

Al hacer esto, aparecerán los eventos al llegar al suscriptor:

```
Event received: Hello World #0
Event received: Hello World #1
Event received: Hello World #2
Event received: Hello World #3
Event received: Hello World #4
Event received: Hello World #5
Event received: Hello World #6
Event received: Hello World #7
Event received: Hello World #8
Event received: Hello World #9
```

Es de destacar el hecho de que los eventos estén modelados como invocaciones. De ese modo es posible eliminar temporalmente el canal de eventos e invocar directamente a un solo suscriptor durante las fases iniciales de modelado del protocolo y desarrollo de prototipos.

4.2 Federación de canales

La federación de canales de eventos representa un mecanismo que, apropiadamente usado, puede aportar grandes ventajas en cuanto a escalabilidad y tolerancia a fallos a un determinado entorno. Generalmente los servicios de eventos asociados a las plataformas distribuidas contemplan este mecanismo para asociar canales de eventos y permitir escalar los entornos de forma que un evento generado en un canal de eventos en particular es transmitido a sus canales federados.

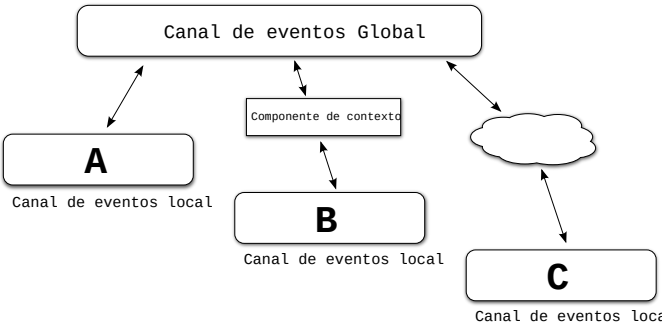


Figure 4.1: Esquema de federación de canales

Se contemplan tres formas de federación posibles:

Federación directa

La federación directa es la más básica y generalmente está soportada por el servicio de eventos del middleware. Este tipo de federación debe ser usada entre canales de eventos del mismo tipo (es decir, mensajes con la misma estructura) y con el objetivo general de agrupar eventos por funcionalidad en un entorno y configurar entornos tolerantes a fallos.

En la figura 4.1, el canal de eventos A se federa al canal de eventos Global por medio del servicio básico. Si el canal de eventos A y el canal de eventos

Global están soportados por distintas instancias del servicio de eventos, una caída del servicio de eventos que da soporte al canal A no afecta al canal de eventos Global y viceversa.

Federación indirecta

Se realiza a través de un componente de contexto. Al poner un elemento intermedio entre los dos canales de eventos se permite la implementación de políticas complejas en el paso de eventos de un canal a otro. En la figura 4.1 el canal de eventos B está conectado a través de un componente de contexto (que es un objeto distribuido) al canal de eventos Global.

Un ejemplo de aplicación de este tipo de federación podría ser la gestión de los sensores de presencia en un entorno laboral (ej: fábricas, oficinas, plantas, etc.). En horario diurno cuando los trabajadores se encuentran realizando su jornada laboral se puede indicar a los componentes de contexto que no retransmitan los eventos de presencia ya que son continuos y están asociados al ir y venir de operarios por la instalación. En horario nocturno esta política cambia y se indica a dichos componentes que retransmitan dichos eventos ya que los mismos eventos de presencia son considerados como eventos de intrusión. Es necesario resaltar que los propios servicios de eventos permiten establecer políticas básicas por lo que este escenario está contemplado para la implementación de políticas relativamente complejas difíciles de abordar con el mecanismo de políticas básicas. Este escenario permite federar canales de eventos de distinto tipo y filtrar la información que debe ser manejada por instancias superiores.

Federación remota

Se considera una federación remota (canal de eventos C en la figura 4.1) cuando los eventos retransmitidos deben atravesar secciones de red no pertenecientes a la organización (ej: a través de Internet) o entornos poco seguros (ej: la vía pública). Esta federación se puede realizar a su vez de forma directa o de forma indirecta (ej: a través de un componente de contexto localizado en el entorno al que pertenece el canal de eventos C). Es necesario resaltar que en estos escenarios se debe establecer una conexión segura (SSL, VPN, Autenticación, etc.) para mantener la confidencialidad de las comunicaciones.

Los mecanismos de federación de canales de eventos deben ser utilizados para segmentar la funcionalidad del entorno atendiendo a razones de despliegue físicas (entornos autónomos y estancos) o de funcionalidad lógica (jerarquizando el paso de mensajes).

4.2.1 Propagación entre canales de eventos federados

La política de propagación de mensajes entre canales de eventos federados es un aspecto crítico para que una arquitectura de eventos sea eficiente y no sobrecargue la red.

Uno de los aspectos mas importantes a la hora de propagar mensajes entre canales de eventos federados es evitar establecer bucles ya sea de forma directa o indirecta que originen mensajes que pasan de un canal a otro de forma recurrente.

Para evitar este tipo de bucles se define el coste de un mensaje. El coste asociado a un mensaje es un valor entero que nos determinará la política de propagación que debemos seguir con ese mensaje.

Para los distintos tipos de federaciones, definimos la siguiente política:

- Los mensajes nunca son propagados mas allá de un solo enlace.
- Los enlaces deben tener un coste asociado, aquellos canales de eventos que tienen otros canales federados, deben propagar un mensaje a dichos canales si el coste que lleva asociado el mensaje es igual o inferior al coste asociado al enlace.
- Si un enlace establece un coste 0, entonces todos los mensajes serán enviados a través de dicho enlace sin importar el coste del mensaje.
- Si un mensaje tiene un coste 0 será enviado a todos los canales de eventos federados sin importar el coste del mensaje.
- Los enlaces entre canales de eventos con un coste 0 deben ser utilizados para difundir de forma automática todos los mensajes de un canal a otro de forma directa (e.j para modelar escenarios mas extensos).
- La federación directa, tal y como podemos ver en la siguiente figura, es útil para modularizar escenarios y agrupar dispositivos que generan el mismo tipo de mensajes en función de su localización física, agrupación funcional, etc.

Chapter 5

Invocación y despachado asíncrono

[Python]

En este capítulo veremos dos técnicas (invocación asíncrona y despachado asíncrono) muy convenientes para reducir el impacto que las comunicaciones pueden suponer para el rendimiento de la aplicación distribuida.

Cuando un cliente invoca un método remoto la ejecución se bloquea hasta que el método ha terminado, de igual forma que ocurre en una invocación a método local convencional. La diferencia es que ésta espera puede durar entre 10 y 100 veces más en una invocación remota que en una local, aparte por supuesto del trabajo real que implique el método. En muchas situaciones el cliente podría realizar alguna otra tarea mientras espera a que la ejecución del método haya concluido. Las técnicas para lograr esto suelen denominarse «programación asíncrona».

Dichas técnicas ayudan a conseguir un sistema más eficiente y escalable, aprovechando mejor los recursos disponibles.

5.1 Invocación síncrona de métodos

Por defecto, el modelo de envío de peticiones utilizado por Ice está basado en la invocación síncrona de métodos remotos: la invocación de la operación se comporta como una llamada a un método local convencional, es decir, el hilo del cliente se bloquea durante la ejecución del método y se vuelve a activar cuando la llamada se ha completado, y todos los resultados están disponibles. La misma situación se da a pesar de que el método no devuelva ningún resultado (como las invocaciones locales).

Éste es el caso de los programas de los capítulos anteriores. Es fácil comprobar que funciona de este modo sin más que añadir una pausa de unos pocos segundos en la implementación del método en el sirviente (tal como proponía el ejercicio **E 1.11**). La ejecución del cliente dura poco más que la pausa que pongamos en el sirviente.

5.2 Invocación asíncrona de métodos

Hay muchas situaciones en las que el cliente puede realizar otras tareas mientras espera a que el objeto remoto realice el trabajo y devuelva los resultados (valor de retorno). O simplemente porque no existen o no necesita dichos resultados. Esto es especialmente relevante en programas con interfaz gráfica. En estos casos se puede conseguir una importante mejora de rendimiento utilizando invocación asíncrona.

Ice permite recoger el resultado de una invocación asíncrona de varias formas:

- Esperarlo más tarde, cuando el cliente lo necesite.
- Registrar un *callback* que se ejecute cuando llegue.
- Comprobar periódicamente si ha llegado (*polling*).

Las dos primeras son las más interesantes y son las que veremos a continuación. El repositorio de ejemplos incluye también un cliente que hace *polling* (`py/ami/client-poll.py`) y otro que integra la invocación en un bucle de eventos de `asyncio` (`py/ami/client-await.py`).

Una cuestión que resulta particularmente interesante es el hecho de que el servidor no está involucrado en forma alguna en los mecanismos Asynchronous Method Invocation (AMI), atañe únicamente a la parte cliente. De hecho, desde el servidor no es posible saber si el cliente está haciendo una invocación asíncrona o síncrona.

5.2.1 Recogiendo el resultado más tarde

El compilador de interfaces genera, para cada operación, una versión asíncrona con el sufijo `Async`. Al invocarla, el cliente obtiene inmediatamente un **future** (`Ice.Future`), un objeto que representa la invocación en curso y que contendrá el resultado (o la excepción) cuando llegue la respuesta. El cliente puede realizar otras operaciones y, cuando necesite el resultado, pedírselo al future con el método `result()`. Aunque la operación no devuelva nada, puede ser interesante hacerlo para comprobar que ha terminado correctamente en el servidor.

En este caso, utilizamos un fichero Slice (ver listado 5.1) que describe la operación `factorial()`, es decir, en este caso si hay un valor de retorno:

Listado 5.1: Especificación Slice para cálculo del factorial (`py/ami/factorial.ice`)

```
module Example {
  interface Math {
    long factorial(int value);
  };
};
```

El listado 5.2 muestra el cliente completo.

Listado 5.2: Cliente AMI que recoge el resultado más tarde (py/ami/client-block.py)

```
1  def main(ic):
2      if len(sys.argv) != 3:
3          print(f"usage: {sys.argv[0]} <proxy> <value>")
4          return 1
5
6      proxy = ic.stringToProxy(sys.argv[1])
7      math = Example.MathPrx.checkedCast(proxy)
8      value = int(sys.argv[2])
9
10     if not math:
11         raise RuntimeError('Invalid proxy')
12
13     future = math.factorialAsync(value)
14     print("That was an async call")
15
16     print(f"Async result is: {future.result()}")
17
18     return 0
```

La parte interesante de este listado comienza en la **línea 13**, en la que el cliente invoca el método `factorialAsync()` del proxy `math` con el valor recibido por línea de comandos y obtiene el `future`.

Después se imprime un mensaje (**línea 14**) que demuestra que el cliente puede realizar otras operaciones mientras la invocación se está realizando. Por último, el método `result()` del future (**línea 16**) devuelve el resultado del cálculo del factorial.

Si en ese momento la invocación no se hubiera completado aún, el cliente quedaría bloqueado en espera de su finalización.

El diagrama de secuencia de la figura 5.1 muestra todo el proceso.

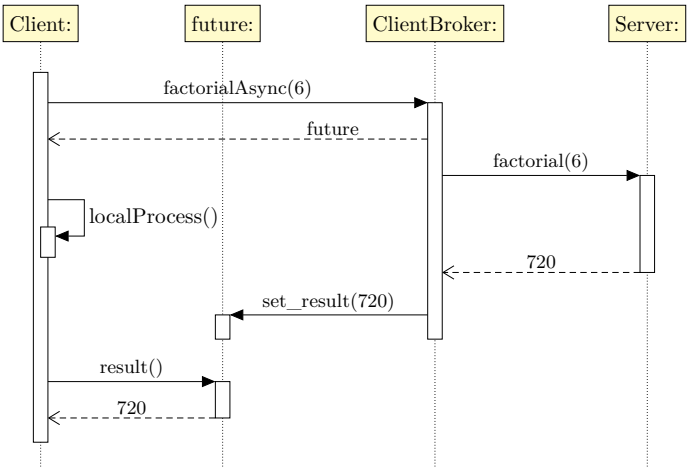


Figure 5.1: Invocación asíncrona (el cliente recoge el resultado más tarde)

Tal como se ha indicado anteriormente, el servidor se implementa de la forma habitual. Se muestra en el listado 5.3.

Listado 5.3: Servidor para `Math.factorial()` (py/ami/server.py)

```python
import sys
import Ice
from pathlib import Path

Ice.loadSlice(str(Path(__file__).parent / 'factorial.ice'))
import Example

def factorial(n):
    if n == 0:
        return 1

    return n * factorial(n - 1)

class MathI(Example.Math):
    def factorial(self, n, current=None):
        return factorial(n)

def main(ic):
    adapter = ic.createObjectAdapter("MathAdapter")
    proxy = adapter.add(MathI(), Ice.stringToIdentity("math1"))

    print(proxy)

    adapter.activate()
    ic.waitForShutdown()
    return 0
```

5.2.2 Utilizando un *callback*

En este caso el cliente no espera el resultado, sino que asocia al future una función de retrollamada (*callback*) que el *runtime* invocará cuando llegue la respuesta. La invocación retorna inmediatamente y el cliente puede seguir ejecutando otras operaciones.

En el listado 5.4 se muestra el código completo para el cliente que crea y proporciona un *callback* según la técnica indicada.

Listado 5.4: Cliente AMI que recibe la respuesta mediante un *callback* (py/ami/client-callback.py)

```python
def callback(future):
    try:
        print(f"Callback: value is: {future.result()}")
    except Exception as ex:
        print(f"Exception is: {ex}")

def main(ic):
    if len(sys.argv) != 3:
        print(f"usage: {sys.argv[0]} <proxy> <value>")
        return 1

    proxy = ic.stringToProxy(sys.argv[1])
```

```
13      math = Example.MathPrx.checkedCast(proxy)
14      value = int(sys.argv[2])
15
16      if not math:
17          raise RuntimeError('Invalid proxy')
18
19      future = math.factorialAsync(value)
20      future.add_done_callback(callback)
21
22      print("That was an async call")
23
24      return 0
```

La llamada asíncrona se realiza en la **línea 19** y en la siguiente se le proporciona el *callback* con el método `add_done_callback()`. Cuando la invocación se complete, el *runtime* local invocará el método `callback()` pasando por parámetro un objeto `Future` con el que se puede acceder al resultado, o elevar la excepción.

El diagrama de secuencia de la figura 5.2 representa las interacciones entre los distintos componentes.

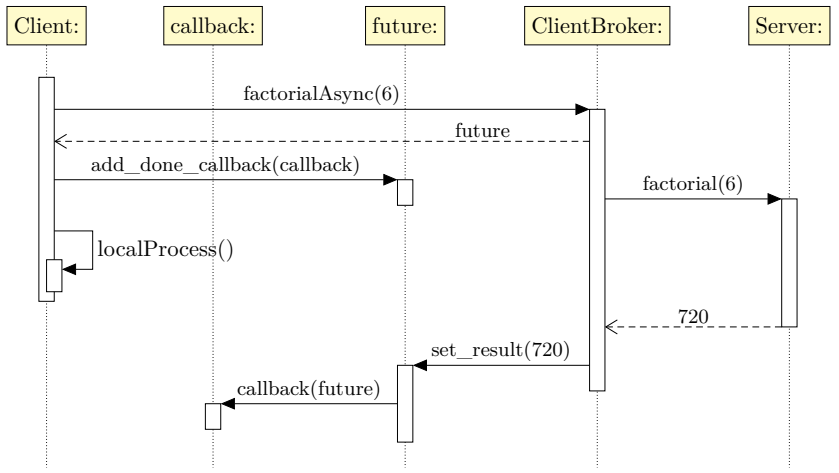


Figure 5.2: Invocación asíncrona (objeto *callback*)

5.3 Despachado asíncrono de métodos

El tratamiento de métodos asíncrono Asynchronous Method Dispatching (AMD) es la contrapartida de AMI en el lado del servidor. En el tratamiento síncrono por defecto, el núcleo de ejecución de la parte servidora ejecuta la operación en el momento en el que recibe. Si esa operación implica utilizar algún recurso adicional, el hilo quedará bloqueado en espera de dicho recurso o de que se completen las operaciones derivadas correspondientes. Eso implica que el hilo de ejecución asociado a la invocación en el servidor sólo se libera cuando la operación se ha completado.

Con AMD se informa al sirviente de la llegada de la invocación. Pero, en lugar de forzar al proceso a atender la petición inmediatamente, el servidor puede almacenar en una cola la especificación de la tarea asociada a esa solicitud y atender la tarea en otro momento. De ese modo se puede liberar el hilo asociado a la invocación.

El servidor puede utilizar otro hilo (que puede ser siempre el mismo si se desea) para extraer las tareas de la cola y procesarla en la forma y tiempo que considere más adecuada en función de los recursos disponibles. Estos hilos, que extraen y procesan tareas de la cola, se les suele denominar *workers* y de hecho puede haber varios sin mayor problema. Cuando los resultados de la tarea están disponibles, el servidor puede recuperar un objeto especial proporcionado por el API del middleware para enviar el mensaje de respuesta al cliente.

AMD es útil, por ejemplo, si un servidor ofrece operaciones que bloquean a los clientes por un periodo largo de tiempo. Por ejemplo, el servidor puede tener un objeto con una operación `get()` que devuelve los datos de una fuente de datos externa y asíncrona, lo cual bloquearía al cliente hasta que los datos estuvieran disponibles.

Con el tratamiento síncrono, cada cliente que espere unos determinados datos estaría vinculado a un hilo de ejecución del servidor. Claramente, este enfoque no es escalable con una docena de clientes. Con AMD, cientos o incluso miles de clientes podrían bloquearse en la misma invocación, pero desacoplados del modelo de concurrencia que utilice el servidor.

El tratamiento síncrono y asíncrono de métodos es transparente al cliente, es decir, el cliente es incapaz de determinar si un cierto servidor realiza un tratamiento síncrono o asíncrono de las invocaciones que recibe.

Para gestionar las tareas pendientes se utiliza una cola en la que se introducen estructuras de datos que incluyen los argumentos de entrada y la instancia que permite recuperar el manejador de la invocación para enviar la respuesta. Esta cola debe ser *thread-safe* dado que las tareas se introducen desde el hilo del sirviente y se extraen desde un *worker* (un hilo diferente).

Para utilizar AMD es necesario añadir metadatos a la especificación Slice. El listado 5.5 muestra el fichero de interfaces que utilizamos en este caso.

Listado 5.5: Especificación Slice para cálculo del factorial con AMD (py/amd/factorial.ice)

```
1  module Example {
2    interface Math {
3      ["amd"] long factorial(int value);
4    };
5
6    exception RequestCanceledException {};
7  };
```

En la **línea 3** se puede apreciar la etiqueta `["amd"]` que le indica al compilador de interfaces que debe generar soporte en el servidor para la gestión de la respuesta

asíncrona tal como se ha descrito. El listado 5.6 muestra el código del servidor AMD.

Listado 5.6: Servidor AMD
(py/amd/server.py)

```
1  import sys
2  import Ice
3  from pathlib import Path
4
5  Ice.loadSlice(str(Path(__file__).parent / 'factorial.ice'))
6  import Example
7
8  from work_queue import WorkQueue
9
10 class MathI(Example.Math):
11     def __init__(self, work_queue):
12         self.work_queue = work_queue
13
14     def factorial(self, value, current=None):
15         future = Ice.Future()
16         self.work_queue.add(future, value)
17         return future
18
19 def main(ic):
20     work_queue = WorkQueue()
21
22     adapter = ic.createObjectAdapter("MathAdapter")
23     proxy = adapter.add(MathI(work_queue), Ice.stringToIdentity("math1"))
24
25     print(proxy)
26
27     adapter.activate()
28     work_queue.start()
29
30     try:
31         ic.waitForShutdown()
32     finally:
33         work_queue.destroy()
34
35     return 0
```

Las diferencias respecto a un servidor convencional están en la creación de la cola de tareas `work_queue` (**línea 20**), que se pasa en el constructor del sirviente (**línea 23**), el arranque de la cola (**línea 28**) y la destrucción de la cola al terminar el programa (**línea 33**).

No se incluye aquí el código de la implementación de la cola de tareas. En todo caso puedes ver su implementación en el repositorio de ejemplos, concretamente en la ruta `py/amd/work_queue.py`.

En este caso el sirviente es extremadamente simple. La implementación del método `factorial()` (**línea 14**) no calcula el resultado: crea un objeto `Ice.Future` (**línea 15**), lo añade a la cola de tareas junto con el parámetro (`value`) y lo devuelve inmediatamente. El núcleo de ejecución enviará la respuesta al cliente cuando la cola complete ese `Future` con el resultado.

Como en los casos anteriores, el proceso se resume en el diagrama de secuencia 5.3.

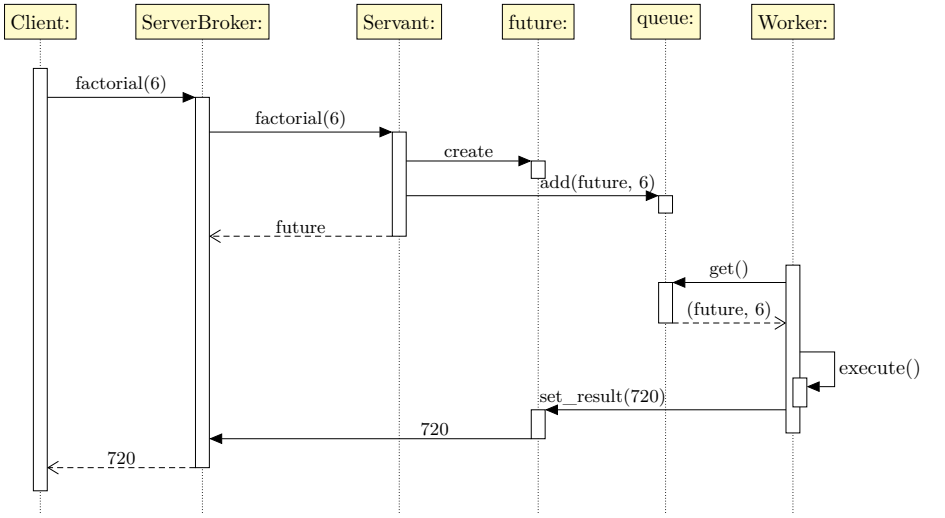


Figure 5.3: Despachado asíncrono

5.4 Mecanismos desacoplados

Es importante recalcar que los mecanismos AMI y AMD están completamente desacoplados y que la decisión de utilizarlos involucra únicamente al programador de cada parte. Es perfectamente posible implementar una o incluso varios técnicas AMI a la vez en el mismo cliente, hacer invocaciones AMI a otros objetos desde un sirviente[1] o en un mismo servidor, utilizar AMD para el despachado y AMI para la invocación a otros objetos.

5.5 Ejercicios

E 5.01 Escribe un cliente AMI y un servidor para demostrar que efectivamente el cliente puede hacer otras operaciones mientras espera a que el servidor termine la invocación.

E 5.02 Escribe un cliente AMI que invoca 10 objetos de forma asíncrona y después recoge los resultados. La invocación al siguiente objeto se realizará antes de recibir los resultados de la invocación anterior.

E 5.03 Con la configuración por defecto un cliente AMI no puede realizar varias invocaciones asíncronas a un mismo objeto. ¿Por qué? ¿Qué cambio se debe hacer para lograrlo?

E 5.04 Escribe un nuevo servidor de *Example.Math* que redirige las invocaciones al servidor original (de forma similar al ejercicio **E 1.13**). Este nuevo

[1] Esta situación requiere configuración específica en el servidor puesto que la ejecución del sirviente se realiza en un hilo, y la invocación AMI necesita un hilo adicional.

servidor debe hacer despachado asíncrono (AMD) para atender a los clientes e invocación asíncrona (AMI) para invocar al servidor original.

E 5.05 La `WorkQueue` implementada en el ejemplo de AMD tiene un único *worker*. Escribe una `WorkQueue` con una cantidad configurable de *workers* atendiendo la misma cola.

Chapter 6

Patrones

[Python]

En cualquier middleware orientado a objetos como Ice, los objetos obviamente son los protagonistas. En este capítulos verás algunas soluciones típicas para cubrir algunas necesidades básicas en la gestión y manipulación de objetos. Los denominados *servicios comunes* (los que ofrece el fabricante con el propio middleware) cubren precisamente necesidades de este tipo.

En este capítulo no se explica como funcionan estos *servicios comunes*. En su lugar, veremos cómo implementar versiones minimalistas alternativas, que si bien pueden ser insuficientes para una aplicación en producción, te servirán para entender cuál es la finalidad y funcionamiento de sus equivalente más sofisticados. Algunos de estos servicios se asemejan (o directamente corresponden) con los patrones de diseño clásicos [GHJV94], mientras que en otros son simplemente soluciones de conveniencia práctica.

6.1 Contenedor de objetos

Un contenedor (o directorio) de objetos es un servicio tremendamente simple. Se trata de un objeto distribuido que almacena referencias a otros objetos cualesquiera. Una interfaz sencilla para este servicio podría ser la siguiente:

Listado 6.1: Especificación Slice para el contenedor de objetos
py/container/container.ice

```
module Services {
  exception AlreadyExists { string key; };
  exception NoSuchKey { string key; };

  dictionary<string, Object*> ObjectPrxDict;

  interface Container {
    void link(string key, Object* proxy) throws AlreadyExists;
    void unlink(string key) throws NoSuchKey;
    Object* get(string key) throws NoSuchKey;
    ObjectPrxDict list();
  };
};
```

La interfaz *Container* tiene cuatro métodos muy simples:

link Añade al directorio una referencia a un objeto arbitrario mediante una clave.

unlink
Elimina una referencia a un objeto enlazado en el contenedor dada su clave.

get Obtiene la referencia a un objeto dada su clave.

list Devuelve un diccionario (una tabla asociativa) de clave-objeto.

Es fácil ver que la implementación del servicio en sí es en esencia una tabla asociativa. Veamos la implementación del sirviente en el listado 6.2.

Listado 6.2: Sirviente del contenedor de objetos
py/container/server.py

```
1  class ContainerI(Services.Container):
2      def __init__(self):
3          self.proxies = dict()
4
5      def link(self, key, proxy, current=None):
6          if key in self.proxies:
7              raise Services.AlreadyExists(key)
8
9          print("link: {0} -> {1}".format(key, proxy))
10         self.proxies[key] = proxy
11
12     def unlink(self, key, current=None):
13         if not key in self.proxies:
14             raise Services.NoSuchKey(key)
15
16         print("unlink: {0}".format(key))
17         del self.proxies[key]
18
19     def get(self, key, current=None):
20         if key not in self.proxies:
21             raise Services.NoSuchKey(key)
22
23         proxy = self.proxies[key]
24         print("get: {0} -> {1}".format(key, proxy))
25         return proxy
26
27     def list(self, current=None):
28         return self.proxies
```

Veamos algunos detalles destacables. Como se puede apreciar, el sirviente efectivamente almacena las referencias en un diccionario (**línea 3**). En la implementación del método `link()` se comprueba si la clave ya existe (**línea 6**), caso en el que se eleva la excepción remota `AlreadyExists`; esto significa que no es posible sobreescribir componentes del contenedor con la misma clave. La contraparte de esta comprobación y su excepción correspondiente se tratan en el método `unlink()` para verificar que el objeto que se desea eliminar existe realmente en el contenedor (**línea 13**). Por último, el método `list()` simplemente devuelve una copia del

atributo diccionario, que será serializado por Ice al tipo correspondiente declarado en el fichero Slice: `ObjectPrxDict`.

E 6.06 En este ejemplo se ha implementado el contenedor como una tabla asociativa usando una clave arbitraria, pero bien podríamos haber utilizado la propia identidad del objeto, obviando estas claves. Crea una versión del contenedor de acuerdo a esta idea.

E 6.07 Otra opción podría ser un contenedor que utilice claves en absoluto, es decir, una simple lista, o un conjunto (sin orden ni repeticiones). Implementa también esta versión del contenedor.

Como el tipo de los proxies en el contenedor es `Object*`, este contenedor puede almacenar **cualquier tipo de objeto Ice** y será el cliente del servicio (como suele ser habitual) el encargado de aplicar el molde (*cast*) al tipo concreto que espera. Este servicio es muy útil cuando una aplicación debe manejar una gran cantidad o arbitraria de objetos. También resulta interesante el hecho de que un contenedor como éste puede enlazar otros contenedores, de modo que se podría crear fácilmente una jerarquía de objetos con un nivel de anidación arbitrario.

Es importante destacar que el contenedor únicamente contiene referencias, es decir, los objetos deben existir en algún otro servidor dentro de la aplicación, y no les afecta en absoluto que sus referencias sean añadidas o eliminadas de éste u otros contenedores.

Como probablemente habrás deducido, Ice ofrece se serie algo similar. Efectivamente gracias a IceGrid y en particular al servicio *Locator* es posible asociar nombres a objetos y obtener fácilmente sus referencias; es lo que conocemos como «objetos bien conocidos». Sin embargo, este tipo de servicio «contenedor» es más adecuado para objetos más efímeros o no tan relevantes para la aplicación. Si se obvian las claves, el contenedor puede verse como un *conjunto* de objetos sin un orden específico.

La implementación que proponemos aquí tiene algunas limitaciones e inconvenientes relevantes.

Persistencia

Las referencias que almacena este contenedor no son persistentes. Si el servidor que lo aloja se reinicia o falla se perderá todo la información. Esta limitación podría resolverse con servicio Freeze de Ice u otro sistema de persistencia equivalente. Recuerde que el objetivo principal de los ejemplos de este capítulo es la simplicidad y no la completitud.

E 6.01 Modifique el sirviente `ContainerI` para añadir soporte de persistencia de modo que, al reiniciarse el servidor, el contenedor tenga los mismos proxies.

Escalabilidad

La interfaz que hemos visto para este servicio no escala adecuadamente. Si el contenedor almacena referencias a cientos o miles de objetos, el método `list()` tratará de devolver toda la información en un solo mensaje, y eso puede no ser posible y desde luego poco eficiente, incluso con relativamente pocos objetos. Este problema normalmente se solventa creando «iteradores» o «paginadores».

El *iterador* es un objeto especial con estado [1]; permite obtener un subconjunto acotado del contenido y después haciendo uso de un método del propio iterador, obtener bloques sucesivos hasta agotar los datos (en este caso los proxies) alojados en el contenedor. Un iterador remoto de este tipo plantea a su vez otros problemas puesto que es complejo para el servidor determinar qué iteradores están en uso. Es sencillo imaginar un ataque Denial of Service (DoS) contra un servicio que ofrece iteradores.

El *paginador* persigue el mismo objetivo, pero no tiene estado. A partir de un *tamaño de página* determinado, el cliente puede solicitar en cualquier momento la cantidad de *páginas* que tiene el contenedor y pedir los datos del contenedor que corresponden una página arbitraria. Esta solución conlleva definir una política de obsolescencia ya que los datos añadidos o eliminados del contenedor deben reflejarse en el resultado de la paginación, es decir, las solicitudes de los clientes deben manejar posibles inconsistencias.

Sin validación

No se realiza ningún tipo de validación sobre las referencias a objetos. Los proxies suministrados al container podrían corresponder a objetos inexistentes o inalcanzables. Esta es una cuestión que puede ser objeto de debate. Por ejemplo, que los objetos no sean accesibles para el contenedor no implica necesariamente que no lo sean para el cliente que los añadió o para aquel que los vaya a obtener. En general, es razonable que este tipo de comprobaciones las realicen los clientes y no el servidor, aunque por supuesto también es una posibilidad a considerar en función de las necesidades de la aplicación.

6.2 Factoría de objetos

En los ejemplos que han aparecido hasta ahora en este documento, los únicos objetos distribuidos que se han manejado corresponden a sirvientes creados por el programador del servidor, típicamente en el programa principal. Es decir, una vez arrancado un servidor, no se crea ni se destruye ningún objeto distribuido.

Sin embargo, en muchas aplicaciones puede surgir la necesidad o la conveniencia de crear nuevos objetos distribuidos cuando la aplicación ya está en marcha. Eso no es ningún inconveniente para el propio servicio. Es posible crear nuevos sirvientes y registrarlos a un adaptador en cualquier momento, incluso desde el cuerpo de otro sirviente.

[1] otro patrón de diseño: el *iterator*

Pero vayamos un poco más lejos: ¿Podría un cliente crear un nuevo objeto distribuido? En principio no, puesto que, por definición, eso lo convertiría en un servidor. Bien, redefinamos la pregunta: ¿Podría un cliente crear un nuevo objeto en un servidor remoto? Obviamente sí. Es el mismo caso del párrafo anterior, pero a iniciativa del cliente.

Una factoría de objetos (ya sea concreta o abstracta [2]) es un objeto que crea otros objetos. Lógicamente en el caso que nos ocupa estamos hablando de una factoría de objetos remotos. Es decir, esto permite a un cliente indicar los argumentos del constructor del sirviente, algo que obviamente no puede hacer él directamente. El listado 6.3 muestra la interfaz Slice (llamada *PrinterFactory*) para una factoría concreta de objetos *PrinterFactory*.

Listado 6.3: Interfaz Slice para la factoría de objetos *PrinterFactory*
py/factory/PrinterFactory.ice

```
#include <printer.ice>

module Example {
  exception FactoryError { string reason; };

  interface PrinterFactory {
    Printer* make(string name) throws FactoryError;
  };
};
```

Veamos una implementación sencilla de esta interfaz. El listado 6.4 muestra el sirviente para la interfaz *PrinterFactory*. En el mismo fichero también se encuentra el sirviente para la clase *Printer*, pero es idéntico al que ya vimos en el capítulo 1, por lo que no lo repetimos aquí.

Listado 6.4: Sirviente para la interfaz PrinterFactory
py/factory/server.py

```
class PrinterFactoryI(Example.PrinterFactory):
    def __init__(self):
        self.objects = {}

    def make(self, name, current=None):
        if name in self.objects:
            return self.objects[name]

        print(f"Creating new printer: '{name}'")

        servant = PrinterI(name)
        proxy = current.adapter.addWithUUID(servant)
        printer = self.objects[name] = Example.PrinterPrx.checkedCast(proxy)

        return printer
```

El argumento `name` del método `make()` será un nombre asociado al objeto a crear y como se puede ver, el valor de retorno es de tipo `Printer*`, es decir, *proxy a Printer*.

[2] Ver patrón de diseño [GHJV94]

Que el valor de retorno pueda ser un proxy no es un detalle menor. Significa que las referencias a objetos remotos son valores de pleno derecho en Slice, que se pasan como argumento y se devuelven como resultado igual que un entero o una cadena. De ahí que el conjunto de objetos accesibles no esté fijado de antemano, que es justamente lo que hace posible una factoría.

Eso da lugar a dos modos de paso de parámetros. Los objetos remotos se pasan *por referencia*: lo que viaja es el proxy, y las invocaciones del receptor llegan al objeto original, que sigue donde estaba. Los demás tipos de Slice —números, cadenas, estructuras, secuencias y diccionarios— se pasan *por valor*: se serializan y el receptor obtiene una copia independiente. Un cliente que recibe un `Printer*` de la factoría no recibe una impresora, sino la manera de llegar hasta ella.

E 6.02 El argumento `name` también se podría usar para indicar la identidad del objeto al registrar el sirviente en el adaptador, en lugar generar un UUID como se ve en el ejemplo. Modifica el método `make()` para que se comporte de este modo.

La factoría guarda los proxies de los objetos creados. Si un cliente solicita un objeto con un nombre ya utilizado, la factoría retorna el objeto existente en lugar de crear otro.

El servidor de la factoría no tiene nada especial (ver listado 6.5); simplemente crea una instancia de *PrinterFactoryI* e imprime el proxy como cualquier servidor Ice básico.

Listado 6.5: Servidor para la factoría de objetos
py/factory/server.py

```
def run(ic):
    servant = PrinterFactoryI()

    adapter = ic.createObjectAdapter("PrinterFactoryAdapter")
    proxy = adapter.add(servant, ic.stringToIdentity("printerFactory1"))

    print(proxy)

    adapter.activate()
    ic.waitForShutdown()

    return 0

if __name__ == '__main__':
    try:
        with Ice.initialize(sys.argv) as communicator:
            sys.exit(run(communicator))
    except KeyboardInterrupt:
        print("\nShutting down server...")
```

Por último, un cliente que utiliza la factoría para crear e invocar objetos se muestra en el listado 6.6.

Listado 6.6: Cliente de la factoría de objetos
py/factory/client.py

```python
import sys
import Ice
Ice.loadSlice('-I. --all PrinterFactory.ice')
import Example

def run(ic, args):
    proxy = ic.stringToProxy(args[1])
    factory = Example.PrinterFactoryPrx.checkedCast(proxy)
    if not factory:
        raise RuntimeError('Invalid proxy')

    printer = factory.make("printer1")
    printer.write('Hello World!')

    return 0

if __name__ == '__main__':
    with Ice.initialize(sys.argv) as communicator:
        sys.exit(run(communicator, sys.argv))
```

Por supuesto, el cliente que solicita la creación del objeto no tiene porqué ser el mismo que lo invoca. El cliente podría enviar el proxy del objeto recién creado por la factoría a un tercer componente (otro servidor) para ser invocado allí.

Contenedor anidado

En la sección 6.1 hablábamos de la posibilidad de crear un contenedor multinivel, es decir, un contenedor que puedo contener objetos arbitrarios, pero también otros contenedores anidados.

Puede ser una buena idea combinar el contenedor con la factoría, es decir, darle al contenedor la posibilidad de crear otros contenedores que automáticamente añade como contenido propio. El listado 6.7 muestra una posible interfaz Slice para este contenedor anidado. El método `createContainer()` crea un nuevo contenedor y devuelve su proxy al cliente. El contenedor recién creado se añade automáticamente al contenedor padre.

Listado 6.7: Interfaz de un contenedor anidado
py/container/nested_container.ice

```
module Services {
  exception AlreadyExists { string key; };
  exception NoSuchKey { string key; };

  dictionary<string, Object*> ObjectPrxDict;

  interface NestedContainer {
    void link(string key, Object* proxy) throws AlreadyExists;
    void unlink(string key) throws NoSuchKey;
    Object* get(string key) throws NoSuchKey;
    ObjectPrxDict list();
    Container* makeContainer(string key);
  };
};
```

Destrucción de objetos

Éste es una cuestión clásica de los sistemas distribuidos. Cuando se le da a los clientes la posibilidad de crear objetos en otra localización aparece un problema no trivial de gestión de recursos. El servidor que realmente aloja el objeto (e instancia posiblemente un sirviente) debería, de un modo u otro, hacerse responsable del ciclo de vida del objeto.

Podríamos pensar en dar al cliente la posibilidad de destruir el objeto cuando no lo necesita, pero el servidor no tiene ninguna garantía de que efectivamente ocurrirá. Incluso un cliente correcto y bienintencionado podría fallar repentinamente o el computador en el que se ejecuta, quedando el objeto «huérfano». Además, el cliente responsable de la creación del objeto podría pasar referencias de ese objeto a otros elementos del sistema de modo que no es sencillo asegurar que realmente no hay ningún potencial usuario del objeto. Este problema se conoce como «recolección de basura distribuida» y no es en absoluto trivial de resolver, pues requiere de mecanismos de captura de estado global.

Ice no incluye ningún recolector de basura distribuido, de modo que la gestión del ciclo de vida queda en manos del programador. La convención habitual es declarar en la propia interfaz una operación `destroy()` cuya implementación retira el sirviente del adaptador, con lo que el momento en que un objeto deja de existir forma parte del diseño de la aplicación y no de la infraestructura.

Otros middlewares toman el camino contrario. Java RMI lleva un conteo de las referencias remotas vivas y concede cada una como un préstamo *lease* de duración limitada que el cliente debe renovar periódicamente; cuando el conteo llega a cero, el objeto se puede recolectar. Eso resuelve el caso del cliente que desaparece sin avisar, a cambio de un tráfico de renovación permanente y de no poder recolectar ciclos de objetos que se referencian entre sí desde nodos distintos.

E 6.03 Realice las modificaciones necesarias, tanto en la declaración Slice como en la implementación del servidor, que permitan al cliente destruir objetos bajo demanda. Aparte de los antes mencionados ¿qué otros problemas acarrea ofrecer esta funcionalidad?

E 6.04 Realice las modificaciones necesarias para que el servidor destruya el objeto (e invalide la referencia del cliente) si deja de recibir invocaciones pasada cierta cantidad de tiempo. ¿Qué problemas puede acarrear esta funcionalidad?

6.3 Factoría IceGrid

IceGrid es a fin de cuentas un gestor de servidores y objetos distribuidos. Es relativamente sencillo utilizar los servicios de IceGrid para crear servidores, no directamente objetos, aunque lógicamente estos servidores van a servir objetos Ice. Por eso, siendo precisos diremos que vamos a crear una factoría de servidores.

IceGrid permite definir plantillas para servidores, que son descripciones en XML de los detalles de un servidor. Veamos una plantilla para servidores de la interfaz *Example.Printer*.

Listado 6.8: Plantilla XML para servidores *Example.Printer*

```
<server-template id="PrinterTemplate">
  <parameter name="name"/>
  <server id="${name}" activation="on-demand" exe="./printer-server.py" pwd=".">
    <properties>
      <property name="Ice.StdOut" value="./printer.out"/>
      <property name="Ice.StdErr" value="./printer.err"/>
      <property name="Ice.ServerIdleTime" value="5"/>
      <property name="Identity" value="${name}"/>
    </properties>
    <adapter name="PrinterAdapter" endpoints="default"
      id="${server}.PrinterAdapter"/>
  </server>
</server-template>
```

Esta factoría —que puedes encontrar en el directorio `hello.ice/icegrid-factory`— utiliza la misma interfaz del Listado 6.3. Lo único que cambia un poco es su uso. El argumento de `make()` ahora se utiliza para fijar el nombre del servidor y la identidad del nuevo objeto *Printer*.

No vamos a analizar en detalle aquí la implementación de la factoría, pero podemos resumir el método `make()` de `icegrid-factory/factory.py` como la siguiente secuencia de pasos:

1. Obtiene una referencia al Registry.
2. Crea una sesión para administrar el Registry. Esto requiere usuario y contraseña, aunque en el ejemplo se utilizan credenciales *dummy*.
3. A partir del nombre indicado, busca el nodo IceGrid en el que se va a crear el nuevo servidor (en el ejemplo fijo como `node1`).
4. Comprueba que el nombre de plantilla indicado (en el ejemplo `PrinterTemplate`) está definido en el Registry.
5. Comprueba si ya existe un servidor con el nombre indicado —el argumento de `make()`— y en ese caso obtiene su proxy. Si no existe, entonces crea el servidor y también devuelve el proxy.

El cliente de ejemplo proporcionado (`icegrid-factory/client.py`) es muy similar al del Listado 6.6. Simplemente invoca 2 veces a `make()`, con `printer1` y `printer2` (vea `icegrid-factory/client.py`). El servidor `printer1` ya está declarado en la aplicación (`icegrid-factory/app.xml`), así que la factoría solo lo arranca si es necesario, mientras que `printer2` no existe y la factoría lo crea a partir de la plantilla.

Si exploras la aplicación con `icegridgui` podrás ver los servidores creados dinámicamente por la factoría (ver Figura 6.1).

Fíjate que al crear servidores en lugar de objetos. Para la misma cantidad de objetos, la necesidad de recursos de esta factoría es sensiblemente mayor que la de la § 6.2. Crear procesos es mucho más costoso que crear objetos dentro de un mismo proceso. Por contra, esta factoría aprovecha las posibilidades que le brinda IceGrid,

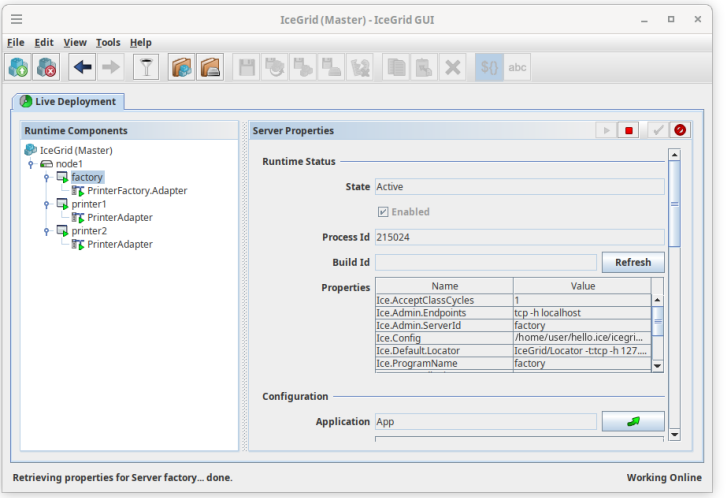


Figure 6.1: Servidores creados dinámicamente por la factoría

como la posibilidad de repartir los servidores en distintos nodos o la desactivación automática de servidores (gracias a la propiedad `ServerIdleTime`).

Decimos que esta es una factoría *concreta* porque solo puede crear servidores que contienen objetos *Printer*. Sin embargo, no es difícil implementar una factoría mucho más flexible que permita no solo crear distintos tipos de objetos (basta con utilizar plantillas distintas), también podría aceptar el nombre de la aplicación, el nodo en el que instanciar el servidor y una lista de parámetros con los que instanciar la plantilla. Esta podría ser una posible interfaz para una factoría abstracta y genérica:

Listado 6.9: Interfaz Slice para la factoría abstracta

```
1  module Service {
2    dictionary<string, string> Parameters;
3    exception FactoryError { string reason; };
4
5    interface ServerFactory {
6      Object* make(string app, string node, string template, Parameters params)
7        throws FactoryError;
8    };
9  };
```

Con este tipo de gestión podemos crear aplicaciones Ice *elásticas* bajo los mismos conceptos que rigen *cloud computing*, es decir, una infraestructura que adapta sus recursos a la demanda, creando y destruyendo servidores en función de la carga, el tráfico o cualquier otro parámetro relevante.

6.4 Default servant

Una alternativa muy interesante a la creación de objetos es el patrón *default servant* (sirviente por defecto). Esta técnica permite que un único sirviente proporcione la funcionalidad de múltiples objetos —potencialmente una cantidad indefinida— siempre que sean del mismo tipo. Los métodos de cualquier sirviente tienen acceso a la identidad del objeto invocado a través de `current.id` y así el sirviente por defecto la puede utilizar para implementar un comportamiento especifico para cada objeto, por ejemplo, a partir de una configuración disponible en una base de datos u otro tipo de persistencia.

El sirviente por defecto se registra en el adaptador de objetos con el método `addDefaultServant()`. Cuando llega una invocación, el adaptador busca primero la tabla de sirvientes (el ASM) registrados de la forma habitual. Si no lo encuentra, el adaptador utiliza entonces el sirviente por defecto.

En realidad se puede registrar un sirviente por defecto por categoría. La categoría es parte de la identidad de un objeto Ice, por ejemplo, la categoría de la identidad `IceGrid/Locator` es `IceGrid`. También es posible especificar una categoría vacía, en cuyo caso solo hay un sirviente por defecto que se aplica a cualquier identidad.

Vamos un servidor sencillo para la interface *Printer*. El sirviente por defecto debe poder determinar de algún modo qué identidades son válidas y elevar la excepción `ObjectNotExistException` en caso contrario. También es necesario sobrecargar el método `ice_ping()` para que los clientes puedan comprobar la existencia de objetos. Esto es lo que vemos en el Listado 6.10.

Listado 6.10: Sirviente por defecto para la interfaz *Printer*
py/default-servant/server.py

```
11 class PrinterI(Example.Printer):
12     def __init__(self, identities):
13         super().__init__()
14         self.valid_identities = set(identities)
15
16     def ensure_valid_object(self, identity):
17         if identity.name not in self.valid_identities:
18             raise Ice.ObjectNotExistException()
19
20     def ice_ping(self, current=None):
21         self.ensure_valid_object(current.id)
22
23     def write(self, message, current=None):
24         self.ensure_valid_object(current.id)
25         print(f"{id2str(current.id)}: {message}")
```

La función `main()` genera 20 identidades, se las pasa como parámetro al sirviente y después crea proxies para cada una de ellas con `createProxy()`. Un cliente convencional puede invocar cualquiera de ellas de la forma habitual. El resto del código del servidor se muestra en el Listado 6.11.

Listado 6.11: Servidor para el sirviente por defecto
py/default-servant/server.py

```
28  def main(ic):
29      identities = [f'printer-{uuid4().hex[:8]}' for _ in range(20)]
30      servant = PrinterI(identities)
31
32      adapter = ic.createObjectAdapter('PrinterAdapter')
33      adapter.addDefaultServant(servant, category='')
34
35      for identity in identities:
36          print(adapter.createDirectProxy(str2id(identity)))
37
38      adapter.activate()
39      ic.waitForShutdown()
40      return 0
```

Chapter 7

Replicación

En Ice, la replicación implica proporcionar (aparentemente) el mismo adaptador (y sus objetos) en varias localizaciones (endpoints). Aquí el principal objetivo de la replicación es proporcionar redundancia, ejecutando varias instancias del mismo servidor en distintos nodos. Si en una de ellas se produce un error, el servicio sigue estando disponible en otros lugares.

Utilizar la replicación requiere que los servidores estén diseñados específicamente. En particular, significa que un cliente observe el mismo comportamiento y obtenga el mismo resultado independientemente de la réplica concreta con la que contacte, ya que el cliente no sabe qué réplica está sirviendo su invocación. Esto es lo que llamamos transparencia de replicación.

El soporte de replicación más básico que proporciona Ice se basa en la posibilidad que tienen los adaptadores de utilizar múltiples endpoints, y funciona incluso con proxies directos y sin necesidad de IceGrid. El núcleo de ejecución de Ice (del cliente) selecciona uno de los endpoints e intenta realizar la conexión. Si falla, el cliente lo intenta con el siguiente endpoint disponible. Es por tanto un mecanismo de *failover* transparente.

Por ejemplo, considera este proxy:

```
1 alarm:tcp -h server1 -p 10001:tcp -h server2 -p 10002
```

Esta referencia indica que el objeto con identidad `alarm` está disponible a través de dos endpoints TCP, uno en el nodo `server1` y otro en el nodo `server2`. Esto podría ocurrir porque el nodo en el que se ejecuta el servidor dispone de dos direcciones IP en la misma red, está conectado a 2 redes diferentes o incluso porque efectivamente el servidor está en ejecución en dos nodos diferentes.

Por defecto, el runtime de Ice reordena aleatoriamente los endpoints del proxy y después los va probando uno a uno hasta que consigue establecer la conexión. Opcionalmente se puede evitar ese reordenamiento y utilizar la lista de endpoints tal como se obtiene. Eso se puede controlar con la propiedad `Ice.Default.EndpointSelection` que puede tomar los valores `random` (por defecto) u `ordered`.

7.1 Grupo de réplicas

Gracias a IceGrid, Ice soporta métodos de replicación más sofisticados basados en el mecanismo de resolución de los proxies indirectos. Este mecanismo se denomina grupo de réplicas (*replica group*). Un grupo de réplicas tiene un identificador único y aglutina un número arbitrario de adaptadores de objetos. Un adaptador de objetos solo puede participar en un grupo de réplicas, en cuyo caso decimos que el adaptador está replicado.

Al crear un proxy indirecto, en lugar del identificador de adaptador habitual, podemos utilizar el identificador del grupo de réplicas. En cierto modo, el grupo de réplicas actúa como un adaptador virtual. Solo en el momento de realizar la invocación, el Locator decide cuál (o cuáles) de los adaptadores reales formarán parte del proxy directo que devuelve al cliente.

Por ejemplo, un grupo de réplicas identificado por `ToolGroup` puede aparecer en un proxy indirecto en la siguiente forma:

```
tool1 @ ToolGroup
```

Veamos el descriptor de una aplicación IceGrid con un grupo de réplicas empleando el tipo más sencillo: *Random*. Lo puedes encontrar en el directorio `replica-group` del repositorio de ejemplos.

Listado 7.1: Aplicación IceGrid con un grupo de réplicas (tipo `random`) (`replica-group/random.xml`)

```
<?xml version="1.0" encoding="UTF-8" ?>
<icegrid>
   <application name="App">
      <server-template id="ToolServerTemplate">
         <parameter name="index"/>
         <server id="ToolServer${index}" activation="always" exe="/app/server.py">
            <properties>
               <property name="Ice.StdOut"/>
               <property name="Ice.StdErr"/>
            </properties>
            <adapter name="ToolAdapter" id="${server}.ToolAdapter"
               endpoints="default" replica-group="ToolGroup"/>
         </server>
      </server-template>
      <node name="node1">
         <server-instance template="ToolServerTemplate" index="1"/>
      </node>
      <node name="node2">
         <server-instance template="ToolServerTemplate" index="2"/>
      </node>
      <replica-group id="ToolGroup">
         <load-balancing type="random" n-replicas="1"/>
      </replica-group>
   </application>
</icegrid>
```

El descriptor primero define una plantilla de servidor (*server-template*) de nombre `ToolServerTemplate` para poder crear fácilmente varios servidores idénticos (solo

cambia su `id`). Eso es lo que aparece después, dos instancias de esa plantilla, en dos nodos. Por último aparece la parte interesante: la definición del grupo de réplicas `ToolGroup`.

```
<replica-group id="ToolGroup">
    <load-balancing type="random" n-replicas="1"/>
</replica-group>
```

Aquí es donde se define el tipo de balanceo de carga y el número de adaptadores que serán incluidos en el proxy directo creado por el Locator. El tipo `random` significa que el Locator elegirá aleatoriamente entre los adaptadores que forman el grupo. El atributo `n-replicas` indica cuántos adaptadores se incluirán en dicho proxy. Si se quiere conseguir transparencia de replicación, es decir, que el cliente no note que el servidor está replicado, el valor debería ser 1, como en el ejemplo.

Aunque el cliente no sepa que está invocando un replica group, podemos saber a qué servidor concreto está invocando realmente fijando la propiedad `Ice.Trace.Locator=1` (fichero `locator.config`). Aquí puedes ver dos invocaciones consecutivas al mismo proxy indirecto `tool1 @ ToolGroup`, y aparece cómo se traduce a dos proxies directos distintos, cada uno con una IP diferente:

```
replica-group$ ../py/upper/client.py --Ice.Config=locator.config "tool1 @ ToolGroup"
-- 12/12/25 18:13:13.744 Locator: retrieved endpoints for adapter from locator
   adapter = ToolGroup
   endpoints = tcp -h 172.28.0.2 -p 44963 -t 60000
HELLO WORLD!
replica-group$ ../py/upper/client.py --Ice.Config=locator.config "tool1 @ ToolGroup"
-- 12/12/25 18:13:14.817 Locator: retrieved endpoints for adapter from locator
   adapter = ToolGroup
   endpoints = tcp -h 172.28.0.3 -p 40055 -t 60000
HELLO WORLD!
```

El tipo `random` favorece el balanceo de carga entre las réplicas. Aunque es muy simple, estadísticamente, todos los nodos recibirán un número similar de invocaciones a largo plazo. Otra forma habitual de conseguir balanceo de carga es `round-robin`, que también soporta IceGrid. Simplemente define el grupo de réplicas así:

Listado 7.2: Aplicación IceGrid con un grupo de réplicas (tipo `round-robin`) (`replica-group/round-robin.xml`)

```
<replica-group id="ToolGroup">
    <load-balancing type="round-robin" n-replicas="1"/>
</replica-group>
```

La replicación tiene otros usos, como por ejemplo aumentar la disponibilidad del servicio, es decir, que el servicio siga estando disponible aunque una o varias réplicas fallen. Eso se puede conseguir con el tipo `ordered`. Cada adaptador se marca con un atributo entero `priority`. El Locator elegirá siempre el adaptador disponible con valor de prioridad menor. Por tanto proporciona un mecanismo de respaldo (*failover*) automático. El descriptor en este caso sería:

Listado 7.3: Aplicación IceGrid con un grupo de réplicas (tipo `ordered`) (`replica-group/ordered.xml`)

```
<?xml version="1.0" encoding="UTF-8" ?>
<icegrid>
   <application name="App">
      <server-template id="ToolServerTemplate">
         <parameter name="index"/>
         <server id="ToolServer${index}" activation="always" exe="/app/server.py">
            <properties>
               <property name="Ice.StdOut"/>
               <property name="Ice.StdErr"/>
            </properties>
            <adapter name="ToolAdapter" id="${server}.ToolAdapter"
               endpoints="default" replica-group="ToolGroup" priority="${index}"/>
         </server>
      </server-template>
      <node name="node1">
         <server-instance template="ToolServerTemplate" index="1"/>
      </node>
      <node name="node2">
         <server-instance template="ToolServerTemplate" index="2"/>
      </node>
      <replica-group id="ToolGroup">
         <load-balancing type="ordered" n-replicas="1"/>
      </replica-group>
   </application>
</icegrid>
```

Del mismo modo que en el caso anterior, puedes comprobar fijando la propiedad `Ice.Trace.Locator=1` en el fichero `locator.config`, que todas las invocaciones se envían a `ToolServer1` (con prioridad 1). Si desactivas ese servidor, las invocaciones que realices a continuación serán atendidas por `ToolServer2`. Si el servidor `ToolServer1` vuelve a estar disponible, las invocaciones se dirigirán de nuevo a él.

Ice proporciona un último tipo llamado `adaptive`, que utiliza una heurística para determinar la carga de los distintos nodos en los que se ejecutan los adaptadores y elegir el menos cargado. Este tipo es más complejo y requiere que los servidores informen periódicamente al Locator de su carga actual. Se define de este modo:

Listado 7.4: Aplicación IceGrid con un grupo de réplicas (tipo `adaptive`) (`replica-group/adaptive.xml`)

```
<replica-group id="ToolGroup">
    <load-balancing type="adaptive" load-sample="5" n-replicas="1"/>
</replica-group>
```

El atributo `load-sample` puede tomar únicamente los valores 1, 5 y 15. Indica el intervalo (en minutos) con el que los nodos IceGrid informan de su carga. Corresponden a los intervalos de muestreo estándar de los sistemas POSIX, que se puede consultar entre otros con el comando `uptime`.

Listado 7.5: Salida del comando `uptime`

```
$ uptime
 22:47:12 up  6:30,  1 user,  load average: 2,23, 1,76, 1,67
```

7.2 Registry replicado

El IceGrid Registry es un componente clave. Es el encargado de activar y monitorizar los servidores IceGrid y también proporciona un servicio esencial: el Locator. Si el Registry o el computador en que se ejecuta falla o queda desconectado de la red, los clientes no podrán contactar con otros objetos y la aplicación dejará de funcionar. Es lo que llamamos un «punto único de fallo» o SPoF.

Para paliar este problema, el Registry proporciona un esquema de replicación maestro-esclavo. Es posible ejecutar varias réplicas del Registry en diferentes nodos. Necesariamente una de ellas se llama `Master` —y actúa como tal—, y las demás serán esclavos —y deben tener nombres únicos. La réplica máster comparte su estado con las réplicas esclavo, de modo que si los clientes pierden el contacto con el máster, pueden consultar a cualquiera de las réplicas. Sin embargo, las réplicas esclavo no ofrecen toda la funcionalidad del Registry, por ejemplo, no pueden desplegar o modificar aplicaciones.

Para conseguir un Registry replicado basta con ejecutar varias instancias del registry y ajustar sus configuraciones. En el directorio `replicated-registry` puedes encontrar un ejemplo con dos réplicas: `Master` y `Slave1`. Lo más destacable de la configuración de `Slave1` son estas líneas:

Listado 7.6: Configuración de la réplica `Slave1` (`replicated-registry/registry2.config`)

```
Ice.Default.Locator=IceGrid/Locator -t:tcp -h registry1 -p 4061:tcp -h registry2 -p 4061
IceGrid.Registry.ReplicaName=Slave1
```

La primera línea (`Ice.Default.Locator`) debería incluir los endpoints de todas las réplicas incluida la que se está configurando. En este caso: `registry1` es el nodo donde se ejecuta el máster y `registry2` es el nodo donde se ejecuta `Slave1`. La segunda línea (`IceGrid.Registry.ReplicaName`) es su nombre.

Los nodos IceGrid al igual que el cliente, también deben incluir la propiedad `Ice.Default.Locator` que incluye los endpoints de todas las réplicas. Por último, el máster por su parte no necesita ningún cambio respecto a una configuración convencional no replicada.

Este ejemplo ejecuta dos réplicas y un nodo en contenedores Docker, así que es fácil comprobar el funcionamiento del mecanismo de *failover*. Una vez está en ejecución, puedes ejecutar el cliente en distintas situaciones parando y arrancando cada uno de los contenedores `registry1` y `registry2`, comprobando que el cliente sigue pudiendo resolver el proxy indirecto y realizar invocaciones.

Referencias

[CDK05] George Coulouris, Jean Dollimore, y Tim Kindberg. *Distributed Systems: Concepts and Design (International Computer Science)*. Addison-Wesley Longman, Amsterdam, edición 4th rev. ed., 2005.

[GHJV94] Erich Gamma, Richard Helm, Ralph Johnson, y John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, edición 1, Noviembre 1994. url: http://www.amazon.com/exec/obidos/redirect?tag=citeulike07-20&path=ASIN/0201633612.

[HS11] M. Henning y M. Spruiell. *Distributed Programming with Ice*. ZeroC Inc., 2011. Revision 3.4.2.